

PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI
KHOA: CÔNG NGHỆ THÔNG TIN
BỘ MÔN: CHUYÊN ĐỀ CÔNG NGHỆ THÔNG TIN

-----o0o-----



BÁO CÁO ĐỀ TÀI
NHẬN DIỆN BỆNH TRÊN LÁ CÂY THỰC VẬT

GVHD: VŨ THỊ HẠNH

SVTH1: NGUYỄN ĐÌNH PHƯỚC CAO

MSSV: 2251068178

SVTH2: Nguyễn Phùng Thịnh

MSSV: 2251068256

SVTH3: Nguyễn Thành Thịnh

MSSV: 2251068257

TP.HCM, NGÀY 31 THÁNG 10 NĂM 2025

MỤC LỤC

Article I.	Tổng quan về bộ dữ liệu & Khám phá dữ liệu:	3
<i>Section I.01</i>	<i>Nguồn bộ dữ liệu: New Plant Diseases Dataset</i>	3
<i>Section I.02</i>	<i>Quy mô tổng thể của bộ dữ liệu:</i>	3
Article II.	Cấu trúc dự án:	23
Article III.	Tiền xử lý dữ liệu	24
Article IV.	Trích xuất đặc trưng:	27
Article V.	5. Huấn luyện mô hình:	31
<i>Section V.01</i>	5.1. Import & logging	31
<i>Section V.02</i>	5.2 Hàm định nghĩa mô hình	32
<i>Section V.03</i>	5.3. Hàm huấn luyện tổng quát	34
<i>Section V.04</i>	4) Hàm trực quan hóa	37
Article VI.	6. Đánh giá mô hình:	44
<i>Section VI.01</i>	Khởi tạo cấu hình	44
<i>Section VI.02</i>	Nạp mô hình trọng số	45
<i>Section VI.03</i>	Biểu đồ báo cáo chi tiết	46
<i>Section VI.04</i>	Bảng so sánh cuối lưu trữ	50
Article VII.	7. Dự đoán bằng hình ảnh	53
<i>Section VII.01</i>	Thiết lập đường dẫn tuyệt đối	55
(a)	Load mô hình đã huấn luyện	58
(b)	Chọn mô hình và layer phù hợp	62

Article I. Tổng quan về bộ dữ liệu & Khám phá dữ liệu:

Section 1.01Nguồn bộ dữ liệu: [New Plant Diseases Dataset](#)

Section 1.02Quy mô tổng thể của bộ dữ liệu:

Thông tin	Số lượng
Tổng số ảnh	87.867 ảnh
Tổng số lớp	38 lớp
Số loại cây	14 loại cây
Tổng số ảnh trong tập train	70.295 ảnh
Tổng số ảnh trong tập valid	17.572 ảnh
Kích thước ảnh ban đầu	256x256
Kích thước ảnh sau thay đổi	224x224
Đặc điểm	Đã tăng cường dữ liệu

a. Phân loại 38 lớp theo loại cây trồng:

STT	Tên Lớp	Loại Cây	Ghi Chú
1	Apple___Apple_scab	Cây táo	Bệnh ghẻ trên cây táo
2	Apple___Black_rot		Bệnh thối đen trên lá cây táo
3	Apple___Cedar_apple_rust		Bệnh rỉ sắt trên cây táo
4	Apple___healthy		Khoẻ mạnh
5	Blueberry___healthy	Cây việt quất	Khoẻ mạnh
6	Cherry___Powery_mildew	Cây sơ-ri	Bệnh phấn trắng
7	Cherry___healthy		Khoẻ mạnh
8	Corn___Cercospora_leaf_spot Gray_leaf_spot	Cây ngô	Bệnh nổi đốm xám trên lá cây
9	Corn___Common_rust		Bệnh rỉ sắt phổ biến

10	Corn___Northern_Leaf_Blight		Bệnh đốm lá bắc
11	Corn___healthy		Khoẻ mạnh
12	Grape___Black_rot	Cây nho	Bệnh thối đen trên lá
13	Grape___Esca_(Black_Measles)		Bệnh sởi đen trên lá
14	Grape___Leaf_blight_(Isariopsis_Leaf_Spot)		Bệnh đốm lá thường xuất hiện trên cây nho
15	Grape___healthy		Khoẻ mạnh
16	Orange___Haunglongbing_(Citrus_greening)	Cam	Bệnh vàng lá gân xanh
17	Peach___Bacterial_spot	Đào	Bệnh đốm khuẩn
18	Peach___healthy		Khoẻ mạnh
19	Pepper_bell___Bacterial_spot	Ớt chuông	Bệnh đốm khuẩn trên lá
20	Pepper_bell___healthy		Khoẻ mạnh
21	Potato___Early_blight	Khoai tây	Bệnh cháy sớm trên lá
22	Potato___Late_blight		Bệnh sương mai trên lá

23	Potato___healthy		Khoẻ mạnh
24	Raspberry___healthy	Phúc bồn tử	Khoẻ mạnh
25	Soybean___healthy	Đậu nành	Khoẻ mạnh
26	Squash___Powdery_mildew	Bí	Bệnh phấn trắng trên lá
27	Strawberry___Leaf_scorch	Dâu tây	Bệnh khô lá cây
28	Strawberry___healthy		Khoẻ mạnh
29	Tomato___Bacterial_spot	Cà chua	Bệnh đốm khuẩn
30	Tomato___Early_blight		Bệnh cháy sớm
31	Tomato___Late_blight		Bệnh sương mai
32	Tomato___Leaf_Mold		Bệnh mốc lá
33	Tomato___Septoria_leaf_spot		Bệnh đốm lá
34	Tomato___Spider_mites Two-spotted_spi- der_mite		Bệnh nhện đỏ tán công trên lá

35	Tomato___Target_Spot		Bệnh đốm mục tiêu trên lá
36	Tomato___Tomato_Yellow_Leaf_Curl_Virus		Bệnh do virus xoắn lá vàng
37	Tomato___Tomato_mosaic_virus		Bệnh do virus khảm
38	Tomato___healthy		Khoẻ mạnh

– Phân loại theo loại cây:

- Cây cà chua: 10 lớp (9 lớp bệnh & 1 lớp khoẻ)
- Cây táo: 4 lớp (3 lớp bệnh & 1 lớp khoẻ)
- Cây sơ-ri: 2 lớp (1 lớp bệnh & 1 lớp khoẻ)
- Cây ngô: 4 lớp (3 lớp bệnh & 1 lớp khoẻ)
- Cây việt quất: 1 lớp khoẻ
- Cây nho: 4 lớp (3 lớp bệnh & 1 lớp khoẻ)
- Cây cam: 1 lớp bệnh
- Cây đào: 2 lớp (1 lớp bệnh & 1 lớp khoẻ)
- Khoai tây: 3 lớp (2 lớp bệnh & 1 lớp khoẻ)
- Ổt chuông: 2 lớp (1 lớp bệnh & 1 lớp khoẻ)
- Phúc bồn tử: 1 lớp khoẻ
- Đậu nành: 1 lớp khoẻ
- Bí: 1 lớp bệnh
- Dâu tây: 2 lớp (1 lớp bệnh & 1 lớp khoẻ)
- Tổng kết:
 - Loại cây có số lượng lớp nhiều nhất: Cà chua
 - Có 26 lớp bệnh + 12 lớp khoẻ

b. Dấu hiệu nhận biết 26 loại bệnh:

STT	Tên tiếng việt	Tên tiếng anh	Dấu hiệu nhận biết	Nguyên nhân
1	Bệnh ghẻ táo	Apple Scab	Vết đen tròn, có xuất hiện nứt nẻ	Nấm
2	Bệnh thối đen ở cây	Apple Black Rot	Vết đen vòng tròn	Nấm

	táo		đồng tâm	
3	Bệnh rỉ sắt táo tuyết tùng	Cedar Apple Rust	Vết vàng cam + chấm đỏ	Nấm
4	Bệnh phấn trắng cây sơ-ri	Cherry Powdery Mildew	Bột trắng mịn	Nấm
5	Bệnh đốm xám ở cây ngô	Corn Gray Leaf Spot	Vết xám hình chữ nhật	Nấm
6	Bệnh rỉ sắt phổ biến ở ngô	Corn Common Rust	Mụn đỏ nâu nổi trên lá	Nấm
7	Bệnh cháy lá phía bắc ở ngô	Corn Northern Leaf Blight	Vết dài xám tro	Nấm
8	Bệnh thối đen ở cây nho	Grape Black Rot	Vết đen lan từ mép lá	Nấm
9	Bệnh đốm đen ở cây nho	Grape Esca (Black Measles)	Lá vàng, vết loang đen	Nấm
10	Bệnh cháy lá ở cây nho	Grape Leaf Blight	Vết nâu viền vàng	Nấm
11	Bệnh vàng lá gân xanh ở cây cam	Orange Haung-longbing (HLB	Lá vàng loang và có gân xanh	Vi khuẩn
12	Bệnh đốm vi khuẩn ở cây đào	Peach Bacterial Spot	Vết nâu ướt, viền vàng	Vi khuẩn
13	Bệnh đốm vi khuẩn ở ớt chuông	Pepper Bacterial	Vết nâu ướt	Vi khuẩn

		Spot		
14	Bệnh mốc sương sớm ở khoai tây	Potato Early Blight	Vết nâu và vòng tròn đồng tâm	Nấm
15	Bệnh mốc sương muộn ở khoai tây	Potato Early Blight	Vết sạm lớn và mốc trắng	Nấm
16	Bệnh phấn trắng ở cây bí	Squash Powdery Mildew	Bột trắng bám lá	Nấm
17	Bệnh cháy lá ở dâu tây	Strawberry Leaf Scorch	Mép lá bị cháy khô	Nấm
18	Bệnh đốm vi khuẩn ở cà chua	Tomato Bacterial Spot	Vết nâu nhỏ và viền vàng	Vi khuẩn
19	Bệnh mốc sương sớm ở cà chua	Tomato Early Blight	Vết nâu và vòng tròn đồng tâm	Nấm
20	Bệnh mốc sương muộn ở cà chua	Tomato Late Blight	Vết xám lớn và mốc trắng	Nấm

21	Bệnh mốc ở lá cây cà chua	Tomato Leaf Mold	Mốc vàng dưới mặt lá	Nấm
22	Bệnh đốm lá ở cây cà chua	Tomato Septoria Leaf Spot	Vết tròn, ở vùng trung tâm có chấm đen	Nấm
23	Bệnh nhện đỏ cây cà chua	Tomato Spider Mites	Chấm vàng li ti	Côn trùng
24	Bệnh đốm mục tiêu ở cây cà chua	Tomato Target Spot	Vết tròn lớn, viền màu nâu	Nấm
25	Bệnh virus khảm lá ở cà chua	Tomato mosaic Virus	Lá xoắn, loang lổ	Virus ToMV
26	Bệnh xoắn vàng lá ở cà chua	Tomato Yellow Leaf Curl Virus	Lá xoắn vàng, cây lùn	Virus TYLCV

– Tổng hợp triệu chứng của lá cây thực vật theo nguyên nhân

Nguyên nhân	Số bệnh	Triệu chứng điển hình	Đặc điểm nhận biết chính	Ví dụ bệnh
-------------	---------	-----------------------	--------------------------	------------

Nấm	19	Vết tròn, có chấm nhỏ, mốc trắng/vàng, bột phấn, cháy ở mép	- Hình dạng (tròn, chữ nhật, có xuất hiện các chấm nhỏ) - Màu sắc: Nâu, xám, đen, vàng - Mốc/phấn : bám trên bề mặt lá	Mốc sương sớm, mốc sương muộn, đốm mục tiêu, phấn trắng, cháy lá phía bắc, đốm xám ngô, cháy lá dâu tây
Vi khuẩn	4	Vết nâu, viền vàng	- Vết nâu nhỏ - Viền vàng sáng - Lan nhanh, lá rụng	Đốm vi khuẩn, đốm vi khuẩn đào, HLB cam
Virus	2	Lá quăn, vết vàng, lá bị biến dạng	- Biến dạng lá (quăn, xoắn, nhỏ) - Màu vàng loang lổ - Không có vết rõ ràng	TYLCV (quăn vàng), ToMV (khảm lá)
Côn trùng	1	Chấm vàng li ti, mạng tơ, lá khô	- Chấm nhỏ li ti - Mạng tơ mỏng - Lá khô vàng, không có mốc	Nhện đỏ

– Tổng kết:

- Bệnh do nấm phổ biến với triệu chứng rõ ràng (vết tròn, chấm nhỏ, mốc trắng/vàng, cháy ở mép) sẽ dễ tách đặc trưng cho mô hình học hỏi và nhận diện.
- Bệnh do vi khuẩn có triệu chứng là vết nâu, viền vàng đặc trưng cũng dễ tách đặc trưng cho mô hình học hỏi và nhận diện
- Bệnh do côn trùng có triệu chứng là chấm vàng li ti, có lớp phủ như mạng tơ, lá khô

- Bệnh do virus riêng biệt có triệu chứng là lá quăn, vệt vàng, lá bị biến dạng.
- Việc phân loại triệu chứng theo nguyên nhân giúp mô hình học hỏi nhanh hơn và dự đoán tương đối chính xác hơn hỗ trợ cho những nông dân chuẩn đoán nhanh tại thực địa

c. Hình ảnh minh họa:

- Bệnh ghẻ ở cây táo (Apple Scab)



Hình 1.1. Bệnh ghẻ ở cây táo

- Bệnh thối đen ở cây táo (Apple Black Rot)



Hình 1.2. Bệnh thối đen ở cây táo

- Bệnh rỉ sắt ở cây táo tuyết tùng (Cedar Apple Rust)



Hình 1.3. Bệnh rỉ sắt ở cây táo tuyết tùng

- Bệnh phấn trắng ở cây anh đào (Cherry Powdery Mildew)



Hình 1.4. Bệnh phấn trắng ở cây anh đào

- Bệnh đốm xám ngô (Corn Gray Leaf Spot)



Hình 1.5. Bệnh đốm xám ở cây ngô

- Bệnh gỉ sắt phổ biến ở cây ngô (Corn Common Rust)



Hình 1.6. Bệnh gỉ sắt phổ biến ở cây ngô

- Bệnh cháy lá phía bắc ở cây ngô (Corn Northern Leaf Blight)



Hình 1.7. Bệnh cháy lá phía bắc ở cây ngô

- Bệnh thối đen ở cây nho (Grape Black Rot)



Hình 1.8. Bệnh thối đen ở cây nho

- Bệnh đốm đen ở cây nho (Grape Esca)



Hình 1.9. Bệnh đốm đen (Esca) ở cây nho

- Bệnh cháy ở lá cây nho (Grape Leaf Blight)



Hình 1.10. Bệnh cháy lá ở cây nho

- Bệnh vàng lá gân xanh ở cây cam (Orange HLB)



Hình 1.11. Bệnh vàng lá gân xanh ở cây cam

- Bệnh đốm vi khuẩn ở cây đào (Peach Bacterial Spot)



Hình 1.12. Bệnh đốm vi khuẩn ở cây đào

- Bệnh đốm vi khuẩn ở cây ớt chuông (Pepper Bacterial Spot)



Hình 1.13. Bệnh đốm vi khuẩn ở cây ớt chuông

- Bệnh mốc sương sớm ở khoai tây (Potato Early Blight)



Hình 1.14. Bệnh mốc sương sớm ở khoai tây

- Bệnh mốc sương muộn ở khoai tây (Potato Late Blight)



Hình 1.15. Bệnh mốc sương muện ở khoai tây

- Bệnh phấn trắng ở cây bí (Squash Powdery Mildew)



Hình 1.16. Bệnh phấn trắng ở cây bí

- Bệnh cháy lá ở dâu tây (Strawberry Leaf Scorch)



Hình 1.17. Bệnh cháy lá ở cây dâu tây

- Bệnh đốm vi khuẩn ở cà chua (Tomato Bacterial Spot)



Hình 1.18. Bệnh đốm vi khuẩn ở cây cà chua

- Bệnh mốc sương sớm ở cà chua (Tomato Early Blight)



Hình 1.19. Bệnh mốc sương sớm ở cây cà chua

- Bệnh mốc sương muộn ở cà chua (Tomato Late Blight)



Hình 1.20. Bệnh mốc sương muộn ở cây cà chua

- Bệnh mốc lá ở cà chua (Tomato Leaf Mold)



Hình 1.21. Bệnh móc lá ở cây cà chua

- Bệnh đốm lá Septoria (Tomato Septoria Leaf Spot)



Hình 1.22. Bệnh đốm lá Septoria ở cây cà chua

- Bệnh nhện đỏ cà chua (Tomato Spider Mites)



Hình 1.23. Bệnh nhện đỏ ở cây cà chua

- Bệnh đốm mục tiêu ở cà chua (Tomato Target Spot)



Hình 1.24. Bệnh đốm mục tiêu ở cây cà chua

- Bệnh virus xoắn vàng lá ở cà chua (Tomato Yellow Leaf Curl Virus)



Hình 1.25. Bệnh virus quăn vàng lá ở cây cà chua

- Bệnh virus khảm lá ở cà chua



Hình 1.26. Bệnh virus khảm lá ở cây cà chua

Article II. Cấu trúc dự án:

```

New_Plant_Diseases_Project/
├── Data
│   ├── New Plant Diseases Dataset(Augmented)\New Plant Diseases
│   │   Dataset(Augmented)
│   │   ├── train
│   │   ├── valid
│   │   └── test

```

- |— features (Lưu trữ các đặc trưng sau khi dùng mô hình sau huấn luyện trích xuất)
- |— models (Lưu trữ các mô hình sau huấn luyện)
- |— plots (Lưu trữ các biểu đồ)
- |— reports (Lưu trữ các file json như: classification_report, eda_summary.json, evaluation_report.json)
- |— src
 - |— app.py
 - |— eda.py
 - |— preprocessing.py
 - |— model_training.py
 - |— feature_engineering.py
 - |— evaluation.py
 - |— update_web_data.py
 - |— run.py
- |— venv
- |— web (lưu trữ giao diện)

- Hướng dẫn chạy sản phẩm:
 - Từ đường dẫn: [victornguyen19072004-oss/NewPlantDiseases](https://github.com/victornguyen19072004-oss/NewPlantDiseases), hãy clone sản phẩm về máy với câu lệnh:
 - git clone đường dẫn
 - cd NewPlantDiseases
 - Chuẩn bị môi trường & các file cần thiết:
 - Làm theo hướng dẫn trong README.md
 - Truy cập & tải 5 file nén của thư mục:
 - Data, models, features, plots
 - Giải nén vào thư mục gốc của dự án
 - Thiết lập môi trường ảo:
 - Vào thư mục dự án, sau đó dùng lệnh sau để thiết lập môi trường ảo:
 - python -m venv venv
 - .\venv\Scripts\activate
 - Khởi chạy toàn bộ hệ thống:
 - Sau khi đã kích hoạt môi trường ảo, hãy chạy lệnh sau:
 - python -m src.run

Article III. Tiền xử lý dữ liệu

- Cài đặt các gói thư viện cần thiết & thiết lập thiết bị thực thi

```

1 import os
2 import torch
3 from torch.utils.data import DataLoader
4 from torchvision import datasets, transforms
5 from sklearn.utils.class_weight import compute_class_weight
6 import numpy as np
7
8 # Thiết lập thiết bị để thực thi (có thể CPU hoặc GPU)
9 def get_device():
10     """Trả về GPU nếu có, còn nếu không thì dùng CPU."""
11     return torch.device('cuda' if torch.cuda.is_available() else 'cpu')

```

- Biến đổi dữ liệu cho tập train & tập valid:

```

1 # Biến đổi dữ liệu của tập train & bổ sung thêm bước tăng cường dữ liệu thực tế
2 def get_transforms(is_train: bool = True):
3     # Tạo pipeline biến đổi ảnh cho tập train và tập valid.
4     if is_train:
5         return transforms.Compose([
6             # Thay đổi độ sáng và độ tương phản của hình ảnh
7             transforms.ColorJitter(brightness=0.2, contrast=0.2),
8             # Xoay mỗi ảnh ngẫu nhiên 30 độ
9             transforms.RandomRotation(30),
10            # Cắt xén và làm nghiêng ảnh
11            transforms.RandomAffine(degrees=0, shear=20),
12            # Làm mờ nhẹ cho hình ảnh
13            transforms.Resize((224, 224)),
14            # Chuyển PIL → Tensor [0, 1]
15            transforms.ToTensor(),
16
17            transforms.RandomApply([transforms.GaussianBlur(kernel_size=3)], p=0.3),
18
19            transforms.RandomErasing(p=0.5, scale=(0.02, 0.33), value='random'),
20            # Chuẩn hóa giúp hội tụ nhanh hơn
21            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
22        ])
23     else:
24         return transforms.Compose([
25             transforms.Resize((224, 224)),
26             transforms.ToTensor(),
27             transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
28        ])

```

- Với tập train sẽ tăng cường dữ liệu thêm khi bổ sung các bước biến đổi ảnh với các điều kiện thực tế
- Với tập valid sẽ chỉ biến đổi thông thường không tăng cường dữ liệu


```

1 # Tải 2 tập là tập train đã được tăng cường dữ liệu & tập valid không tăng cường dữ liệu
2 def load_datasets(train_dir: str, valid_dir: str):
3     """Tải dữ liệu cho tập train đã được biến đổi và tăng cường
4     Tải dữ liệu cho tập valid nhưng chỉ biến đổi không tăng cường
5     """
6     train_ds = datasets.ImageFolder(train_dir, transform=get_transforms(True))
7     valid_ds = datasets.ImageFolder(valid_dir, transform=get_transforms(False))
8     return train_ds, valid_ds
9
10 # Tải 1 tập dữ liệu là tập test ( nếu dùng được )
11 def load_single_dataset(dir_path: str):
12
13     if not dir_path:
14         return None
15     dataset = datasets.ImageFolder(dir_path, transform=get_transforms(False))
16     return dataset

```

- Tải tập train & valid từ bộ dữ liệu, và tải đơn dùng cho trường hợp tập test.
 - o Tập valid dùng để điều chỉnh các tham số cho quá trình huấn luyện mô hình và cũng dùng cho việc huấn luyện mô hình tránh bị Overfitting.
 - Khi huấn luyện mô hình thì sự thay đổi của valid loss sẽ làm cho learning rate, batchsize,... Thay đổi
 - Dùng cho việc dừng sớm khi độ chính xác trên tập valid không cải thiện nữa
 - So sánh hiệu suất của các mô hình khác nhau
 - o Tập test dùng cho giả lập môi trường thực tế, dùng khi dự đoán hình ảnh cuối cùng.
- Đóng gói tập train và tập valid vào DataLoaders dùng khi huấn luyện mô hình.
- Tính trọng số của các lớp để tạo ra một bộ trọng số phạt cho phân hàm mất mát (Loss Function) nhằm xử lý tình trạng mất cân đối dữ liệu, lớp nào càng ít mẫu -> Trọng số càng cao.

```

1 # Tính trọng số của lớp
2 def compute_class_weights(train_ds):
3     """Tính trọng số lớp để xử lý imbalance.
4     Tính trọng số lớp theo công thức 'balanced'
5     Dùng trong CrossEntropyLoss
6     Theo cơ chế: Lớp nào ít mẫu thì trọng số cao -> Mô hình sẽ quan tâm nhiều hơn
7     """
8     labels = [label for _, label in train_ds]
9     weights = compute_class_weight('balanced', classes=np.unique(labels), y=labels)
10    return torch.tensor(weights, dtype=torch.float)

```

- Trích xuất các nhãn thực tế, khi hàm duyệt qua toàn bộ tập train và thu thập tất cả các nhãn số nguyên (int labels) ứng với từng lớp trong tập train.
- Tính toán trọng số weights:
 - o Phần tính toán trọng số này thực hiện theo công thức 'balanced':
 - **Trọng số cho lớp i** = (Tổng số ảnh trong tập dữ liệu) / ((tổng số ảnh của tập dữ liệu) x (Số lượng ảnh của lớp thứ i))
 - o Nếu một lớp có số lượng ảnh chiếm đa số, nó sẽ có trọng số thấp
 - o Nếu một lớp có số lượng ảnh chiếm thiểu số, nó sẽ có trọng số cao.
- Chuyển mảng trọng số thành một **Tensor Pytorch** là một tham số khi huấn luyện mô hình với CrossEntropyLoss.
- Hàm mất mát sử dụng CrossEntropyLoss sẽ tính ra giá trị mất mát cho mô hình, nếu mô hình dự đoán sai một số lượng ảnh của lớp thiểu số, giá trị mất mát sẽ được tăng lên và tạo ra một tín hiệu để mô hình tập trung đến
- Nếu mô hình dự đoán sai một số lượng ảnh của lớp đa số, giá trị mất mát sẽ được giữ ở mức thấp, để không tạo tín hiệu gây chi phối quá trình mô hình học.

Article IV. Trích xuất đặc trưng:

- Trước khi trích xuất đặc trưng cần phải thực hiện bước:
 - o Tiền xử lý dữ liệu & huấn luyện mô hình

```

1 try:
2     from src.preprocessing import get_device, load_datasets, get_data loaders
3     from src.model_training import define_resnet18, define_mobilenetv2
4     from torchvision.models import efficientnet_b0
5 except ImportError as e:
6     logging.error(f"Lỗi: Không thể import từ preprocessing hoặc model_training: {e}")
7     raise
8
9 logging.basicConfig(level=logging.INFO)

```

- Lấy mô hình CNN để minh họa cho luồng hoạt động:

```

1 def define_cnn(num_classes: int = 38) → nn.Module:
2     """Hàm định nghĩa mô hình CNN thuần đơn giản"""
3     model = nn.Sequential(
4         # Tạo lớp tích chập thứ 1: 3 kênh màu RGB, 32 ánh xạ đặc trưng, giữ kích thước hình ảnh
5         nn.Conv2d(3, 32, kernel_size=3, padding=1),
6         # Chuẩn hóa activation dùng cho huấn luyện được ổn định
7         nn.BatchNorm2d(32),
8         # RELU phi tuyến
9         nn.ReLU(),
10        # Giảm chiều không gian từ 224 → 112
11        nn.MaxPool2d(2, 2),
12
13        # Tạo lớp tích chập thứ 2: Tăng ánh xạ đặc trưng từ 32 → 64
14        nn.Conv2d(32, 64, kernel_size=3, padding=1),
15        # Chuẩn hóa activation dùng cho huấn luyện được ổn định
16        nn.BatchNorm2d(64),
17        # Phi tuyến tính
18        nn.ReLU(),
19        # Giảm chiều không gian từ 112 → 56
20        nn.MaxPool2d(2, 2),
21
22        # Tạo lớp tích chập thứ 3: Tăng ánh xạ đặc trưng từ 64 → 128
23        nn.Conv2d(64, 128, kernel_size=3, padding=1),
24        nn.BatchNorm2d(128),
25        nn.ReLU(),
26        # Giảm chiều không gian từ 56 → 28
27        nn.MaxPool2d(2, 2),
28
29        # Tạo lớp tích chập thứ 4: Tăng ánh xạ đặc trưng từ 128 → 256
30        nn.Conv2d(128, 256, kernel_size=3, padding=1),
31        nn.BatchNorm2d(256),
32        nn.ReLU(),
33        # Giảm chiều không gian từ 28 → 14
34        nn.MaxPool2d(2, 2),
35
36        # Giảm tham số, kháng over-fitting, thay theescho phần fully-connected
37        nn.AdaptiveAvgPool2d(1),
38        nn.Flatten(),
39
40        # Loại bỏ ngẫu nhiên 50% nơ-ron để kháng over-fitting
41        nn.Dropout(0.5),
42        nn.Linear(256, num_classes)
43    )
44    logging.info("Đã định nghĩa mô hình CNN thuần (Tối ưu GAP).")
45    return model

```

- Khi một bức hình đi qua một lớp tích thì sẽ lọc ra vị trí và đặc điểm của đặc trưng thành **bản đồ đặc trưng**.
 - **Bộ lọc 1:** Phát hiện cạnh ngang có điểm sáng -> Lưu trữ thành bản đồ đặc trưng
 - **Bộ lọc 2:**
 - **Bộ lọc 3:**
 - **Bộ lọc càng nhiều thì mô hình được nhiều đặc điểm của đặc trưng hơn.**

Bước	Đầu vào	Xử lý	Đầu ra	Kích thước
1	Ảnh RGB	Lớp tích chập đầu tiên + chuẩn hoá + ReLU	32 bản đồ đặc trưng cơ bản	224x224x32

			(cạnh, màu, cấu trúc)	
2	224x224x32	MaxPool2d(2)	Giảm kích thước và giữ đặc trưng nổi bật	112x112x32
3	112x112x32	Lớp tích chập thứ 2 + chuẩn hoá + ReLU	64 bản đồ đặc trưng phức tạp hơn	112x112x64
4	112x112x64	MaxPool2d(2)	Giảm kích thước và giữ đặc trưng nổi bật	56x56x64
5	56x56x64	Lớp tích chập thứ 3 + chuẩn hoá + ReLU	128 bản đồ đặc trưng cấp cao: hình dạng của triệu chứng bệnh, vùng triệu chứng xuất hiện	56x56x128
6	56x56x128	MaxPool2d(2)	Giảm kích thước và giữ đặc trưng nổi bật	28x28x128
7	28x28x128	Lớp tích chập thứ 4 + chuẩn hoá + ReLU	256 bản đồ đặc trưng cho một loại bệnh cụ thể	28x28x256
8	28x28x256	MaxPool2d(2)	Giảm kích thước và giữ đặc trưng nổi bật	14x14x256
9	14x14x256	AdaptiveAvgPool2d(1)	Lấy trung	1x1x256

			bình của mỗi bản đồ gộp thành 1 giá trị	
10	1x1x256	Flatten()	Biến bản đồ đặc trưng là ma trận 2x2 thành vec- tor đặc trung	256
11	Vector 256	Dropout(0.5)	Ngẫu nhiên bỏ 50% nơ- ron để hạn chế overfit- ting	256

- Khi sử dụng các mô hình để trích xuất đặc trưng thành vector đặc trưng thì sử dụng các lớp tích chập để có các bản đồ đặc trưng gồm các thông tin về đặc trưng và chuyển về vector 1 chiều lúc này là vector đặc trưng. Mà khi dùng thì không có lớp phân loại.
- Tải các mô hình đã được huấn luyện để trích xuất đặc trưng:

```

1 # tải các mô hình và dùng mô hình để trích xuất đặc trưng
2 def get_feature_extractor_backbone(model_name: str, num_classes: int = 38) → nn.Module:
3     """
4     Tải mô hình (hoặc pre-trained) và cắt bỏ lớp phân loại cuối cùng (Classifier)
5     để chỉ giữ lại bộ trích xuất đặc trưng (Backbone).
6     """
7     # ... (Các logic tách Backbone giữ nguyên) ...
8     if model_name == 'ResNet18':
9         model = define_resnet18(num_classes, use_pretrained=False)
10        backbone = nn.Sequential(*list(model.children())[:-1], nn.Flatten())
11        logging.info("Trích xuất đặc trưng ResNet18 thành công.")
12        return backbone
13
14    elif model_name == 'MobileNetV2':
15        model = define_mobilenetv2(num_classes, use_pretrained=False)
16        # MobileNetV2 features là backbone, AdaptiveAvgPool2d(1) + Flatten chuyển thành vector
17        return nn.Sequential(model.features, nn.AdaptiveAvgPool2d(1), nn.Flatten())
18
19    elif model_name == 'EfficientNet-B0':
20        model = efficientnet_b0(weights=None)
21        # EfficientNet-B0 có 3 lớp cuối cùng là avgpool, classifier
22        backbone = nn.Sequential(*list(model.children())[:-1], nn.AdaptiveAvgPool2d(1), nn.Flatten())
23        logging.info("Tách EfficientNet-B0 Backbone thành công.")
24        return backbone
25
26    elif model_name == 'CNN':
27        # Đối với mô hình tự định nghĩa, chúng ta cần cắt bỏ lớp phân loại cuối cùng
28        model = define_cnn(num_classes)
29        # Cắt bỏ lớp Linear (index -1) và lớp Dropout (index -2)
30        backbone = nn.Sequential(*list(model.children())[:-2])
31        logging.info("Tách CNN Backbone thành công.")
32        return backbone
33
34    else:
35        raise ValueError(f"Mô hình {model_name} không được hỗ trợ để trích xuất đặc trưng.")
36

```

- Tải tập train và tập valid từ DataLoader để mô hình trích xuất đặc trưng

```

1 # Sử dụng mô hình trích xuất đặc trưng
2 def extract_features(model: nn.Module, dataloader: Any, device: torch.device) → Tuple[torch.Tensor, torch.Tensor]:
3     """Hàm chạy qua DataLoader để trích xuất vector đặc trưng và nhãn tương ứng."""
4     model.eval()
5     model.to(device)
6     features = []
7     labels = []
8     logging.info("Bắt đầu trích xuất đặc trưng...")
9
10    with torch.no_grad():
11        for images, lbls in dataloader:
12            images = images.to(device)
13            feature_vector = model(images)
14            features.append(feature_vector.cpu())
15            labels.append(lbls)
16
17    features = torch.cat(features, dim=0)
18    labels = torch.cat(labels, dim=0)
19    logging.info(f"Đã trích xuất xong. Feature vector shape: {features.shape}")
20    return features, labels

```

- Lưu các vector đặc trưng thành file .numpy vào folder features

```

1 # 3. Lưu vector đặc trưng và nhãn thành file .numpy
2 def save_features(features: torch.Tensor, labels: torch.Tensor, model_name: str, save_dir: str = 'features'):
3     """Lưu vector đặc trưng và nhãn ra file numpy để dùng cho mô hình ML."""
4     os.makedirs(save_dir, exist_ok=True)
5     feature_path = os.path.join(save_dir, f'features_{model_name}.numpy')
6     label_path = os.path.join(save_dir, f'labels_{model_name}.numpy')
7
8     np.save(feature_path, features.numpy())
9     np.save(label_path, labels.numpy())
10    logging.info(f"Đã lưu đặc trưng tại {feature_path} và nhãn tại {label_path}")
11
12

```

Article V. 5. Huấn luyện mô hình:

Section 5.01 5.1. Import & logging

```

3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 import matplotlib.pyplot as plt
7 from torchvision import models
8 import logging
9 import os
10 from typing import Dict, Any, Tuple, List

12 logging.basicConfig(level=logging.INFO)

```

`logging.basicConfig(level=logging.INFO)` để in log chuẩn trong quá trình huấn luyện.

Tác dụng: có đủ công cụ để định nghĩa mô hình, train, vẽ biểu đồ và theo dõi tiến trình.

Section 5.02 5.2 Hàm định nghĩa mô hình

```
16 def define_cnn(num_classes: int = 38) -> nn.Module:
17     """Hàm định nghĩa mô hình CNN thuần đơn giản"""
18     model = nn.Sequential(
19         # Tạo lớp tích chập thứ 1: 3 kênh màu RGB, 32 ánh xạ đặc trưng, giữ kích thước hình ảnh
20         nn.Conv2d(3, 32, kernel_size=3, padding=1),
21         # Chuẩn hóa activation dùng cho huấn luyện được ổn định
22         nn.BatchNorm2d(32),
23         # RELU phi tuyến
24         nn.ReLU(),
25         # Giảm chiều không gian từ 224 -> 112
26         nn.MaxPool2d(2, 2),
27
28         # Tạo lớp tích chập thứ 2: Tăng ánh xạ đặc trưng từ 32 -> 64
29         nn.Conv2d(32, 64, kernel_size=3, padding=1),
30         # Chuẩn hóa activation dùng cho huấn luyện được ổn định
31         nn.BatchNorm2d(64),
32         # Phi tuyến tính
33         nn.ReLU(),
34         # Giảm chiều không gian từ 112 -> 56
35         nn.MaxPool2d(2, 2),
36
37         # Tạo lớp tích chập thứ 3: Tăng ánh xạ đặc trưng từ 64 -> 128
38         nn.Conv2d(64, 128, kernel_size=3, padding=1),
39         nn.BatchNorm2d(128),
40         nn.ReLU(),
41         # Giảm chiều không gian từ 56 -> 28
42         nn.MaxPool2d(2, 2),
43
44         # Tạo lớp tích chập thứ 4: Tăng ánh xạ đặc trưng từ 128 -> 256
45         nn.Conv2d(128, 256, kernel_size=3, padding=1),
46         nn.BatchNorm2d(256),
47         nn.ReLU(),
48         # Giảm chiều không gian từ 28 -> 14
49         nn.MaxPool2d(2, 2),
50
51         # Giảm tham số, kháng over-fitting, thay theescho phần fully-connected
52         nn.AdaptiveAvgPool2d(1),
53         nn.Flatten(),
54
55         # Loại bỏ ngẫu nhiên 50% nơ-ron để kháng over-fitting
56         nn.Dropout(0.5),
57         nn.Linear(256, num_classes)
58     )
59     logging.info("Đã định nghĩa mô hình CNN thuần (Tối ưu GAP).")
60     return model
```

a) `define_cnn(num_classes=38) → nn.Module`

-Xây CNN “thuần”: 4 block Conv2d → BatchNorm2d → ReLU → MaxPool2d, kênh tăng dần 3→32→64→128→256.

-Dùng `AdaptiveAvgPool2d(1) + Flatten` thay FC nhiều tầng để giảm tham số.

- Dropout (0.5) trước Linear (256, num_classes) để kháng overfitting.
- Log: “Đã định nghĩa mô hình CNN thuần (Tối ưu GAP)”.

model_training

-Tác dụng: baseline nhẹ, huấn luyện từ đầu (không cần pretrain), có BatchNorm + Dropout nên ổn định/giảm overfit.

```

62 def define_resnet18(num_classes: int = 38, use_pretrained: bool = True) -> nn.Module:
63     """Hàm định nghĩa mô hình ResNet18 pre-trained."""
64     # Tải trọng số ImageNet
65     weights = models.ResNet18_Weights.IMAGENET1K_V1 if use_pretrained else None
66     # Tải kiến trúc của ResNet18
67     model = models.resnet18(weights=weights)
68     num_ftrs = model.fc.in_features
69     model.fc = nn.Linear(num_ftrs, num_classes)
70     logging.info("Đã define mô hình ResNet18.")
71     return model
72

```

b) define_resnet18(num_classes, use_pretrained=True)

-Lấy weights = ResNet18_Weights.IMAGENET1K_V1 khi use_pre-trained.

-Tải kiến trúc models.resnet18(weights=weights).

-Lấy num_ftrs = model.fc.in_features, thay model.fc = nn.Linear(num_ftrs, num_classes).

-Log: “Đã define mô hình ResNet18.”

Tác dụng: Transfer Learning giúp hội tụ nhanh, chính xác cao trên dataset y khoa vừa/nhỏ.

```

73 def define_mobilenetv2(num_classes=38, use_pretrained=True):
74     """Hàm định nghĩa mô hình MobileNetV2 pre-trained."""
75     if use_pretrained:
76         weights = models.MobileNet_V2_Weights.IMAGENET1K_V1
77         model = models.mobilenet_v2(weights=weights)
78     else:
79         model = models.mobilenet_v2(weights=None)
80
81     num_ftrs = model.classifier[1].in_features
82     model.classifier[1] = nn.Linear(num_ftrs, num_classes)
83     logging.info("Đã define mô hình MobileNetV2.")
84     return model
85

```

c) define_mobilenetv2(num_classes=38, use_pretrained=True)

-Lấy MobileNet_V2_Weights.IMAGENET1K_V1 nếu pretrain.

-Thay model.classifier[1] = nn.Linear(num_ftrs, num_classes).

-Log tương ứng.

-Tác dụng: mô hình nhẹ, nhanh, hợp chạy thật trên mobile/edge device.

```
86 def define_efficientnetb0(num_classes: int = 38, use_pretrained: bool = True) -> nn.Module:
87     """Hàm định nghĩa mô hình EfficientNet-B0 pre-trained."""
88     if use_pretrained:
89         weights = models.EfficientNet_B0_Weights.IMAGENET1K_V1
90         model = models.efficientnet_b0(weights=weights)
91     else:
92         model = models.efficientnet_b0(weights=None)
93
94     num_fts = model.classifier[1].in_features
95     model.classifier[1] = nn.Linear(num_fts, num_classes)
96     logging.info("Đã define mô hình EfficientNet-B0.")
97     return model
98
```

d) `define_efficientnetb0(num_classes=38, use_pretrained=True)`

-Tải `efficientnet_b0` + thay `classifier[1]` bằng `Linear(num_fts, num_classes)`.

-Log tương ứng.

-Tác dụng: cân bằng tốt giữa chính xác và tốc độ; baseline rất phổ biến.

Section 5.03 5.3. Hàm huấn luyện tổng quát

```
101 def train_model(
102     model: nn.Module, model_name: str, train_loader: Any, valid_loader: Any,
103     class_weights: torch.Tensor, device: torch.device,
104     epochs: int = 50, lr: float = 0.01, patience: int = 5
105 ) -> Tuple[List[float], List[float], List[float], List[float]]:
```

`model`: mô hình PyTorch (CNN thuần, ResNet18,...).

`model_name`: để đặt tên file checkpoint/biểu đồ.

`train_loader, valid_loader`: luồng batch ảnh/nhân (đã transform).

`class_weights`: trọng số lớp (xử lý mất cân bằng). Thường tính từ `compute_class_weights`.

`device`: cuda hoặc cpu.

`epochs`: số vòng lặp qua toàn bộ train set.

`lr`: (lưu ý) tốc độ học *được truyền vào*, nhưng bên dưới đang không dùng (optimizer hard-code $1e-4$ – nên sửa, xem mục 8).

`patience`: số epoch “nhẫn nại” trước khi early stopping.

Giá trị trả về: 4 danh sách lịch sử: `train_losses`, `valid_losses`, `train_accs`, `valid_accs` → dùng để vẽ learning curves/báo cáo.

```
107     """Hàm huấn luyện mô hình"""
108     model_path = f'models/best_{model_name}.pth'
109     os.makedirs(os.path.dirname(model_path) or '.', exist_ok=True)
110
111
112     # Hàm mất mát: Sử dụng CrossEntropy + xử lý mất cân đối dữ liệu
113     criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
```

Tạo thư mục models/ nếu chưa có, checkpoint sẽ lưu ở đây.

```
115     # Sử dụng Adam để tính toán tốc độ học ( learning-rate ) cho mỗi tham số
116     # Sử dụng L2 regularization để kháng over-fitting
117     optimizer = optim.Adam(model.parameters(), lr=1e-4, weight_decay=1e-4) # Dùng LR thấp cho Trans
118
119     # Giảm tốc độ học xuống nếu như chỉ số mất mát của tập valid không tăng
120     scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'min', patience=3, factor=0.5)
```

-**Adam**: tối ưu theo moment (muộn hơn SGD), hợp với transfer learning.

-`weight_decay=1e-4` = **L2 regularization** → hạn chế overfitting bằng cách “phạt” trọng số lớn.

-**ReduceLROnPlateau**: khi `val_loss` **không giảm** trong 3 epoch, **giảm LR 1/2** → giúp thoát “cao nguyên” tối ưu.

-**Lưu ý**: `lr` truyền vào hàm **không được dùng** (hard-code `1e-4`). Xem mục 8 để fix.

```
122     train_losses, valid_losses, train_accs, valid_accs = [], [], [], []
123     best_valid_loss = float('inf')
124     patience_counter = 0
125     model.to(device)
```

- Lưu lịch sử để vẽ đồ thị.

- `best_valid_loss` dùng cho **early stopping & lưu checkpoint tốt nhất**.

- `patience_counter` đếm số epoch “không cải thiện”.

```
130     model.train()
131     running_loss, correct, total = 0.0, 0, 0
132     for images, labels in train_loader:
133         # Chuyển dữ liệu sang GPU
134         images, labels = images.to(device), labels.to(device)
135
136         # Xóa Gradient cũ nếu có
137         optimizer.zero_grad()
138         outputs = model(images)
139
140         # Tính hàm mất mát
141         loss = criterion(outputs, labels)
142         # Tính gradient
143         loss.backward()
144         # Cập nhật các trọng số
145         optimizer.step()
146         # Ghi nhận các thống kê
147         running_loss += loss.item()
148         _, predicted = torch.max(outputs, 1)
149         total += labels.size(0)
150         correct += (predicted == labels).sum().item()
151
152     # Tính toán trung bình của mỗi epochs
153     train_loss = running_loss / len(train_loader)
154     train_acc = 100 * correct / total
155     train_losses.append(train_loss)
156     train_accs.append(train_acc)
```


- `model.train()`: bật **dropout, batchnorm** ở chế độ train.
- `zero_grad` → `forward` → `loss` → `backward` → `step`: **quy trình tối ưu chuẩn**.
- Tính **epoch train loss** bằng **trung bình loss các batch**; **accuracy** bằng số đúng / tổng mẫu (đổi sang %).

```

158         # --- Phần đánh giá ---
159         model.eval()
160         val_loss, val_correct, val_total = 0.0, 0, 0
161         with torch.no_grad():
162             for images, labels in valid_loader:
163                 images, labels = images.to(device), labels.to(device)
164                 outputs = model(images)
165                 loss = criterion(outputs, labels)
166                 val_loss += loss.item()
167                 _, predicted = torch.max(outputs, 1)
168                 val_total += labels.size(0)
169                 val_correct += (predicted == labels).sum().item()
170
171         val_loss = val_loss / len(valid_loader)
172         val_acc = 100 * val_correct / val_total
173         valid_losses.append(val_loss)
174         valid_accs.append(val_acc)

```

- `model.eval() + torch.no_grad()`: **tắt gradient** để giảm RAM/VRAM & nhanh hơn.
- Tính tương tự train: **loss trung bình, accuracy (%)**.
- **Lưu ý chọn metric chuẩn để dừng sớm**: ở đây dùng `val_loss` làm “tiêu chuẩn vàng” (ổn).

```

176         current_lr = optimizer.param_groups[0]['lr']
177         logging.info(f"Epoch {epoch+1}/{epochs}: Train Loss {train_loss:.4f}, Acc {train_acc:.2f}%")
178
179         # Cập nhật tham số tốc độ học (learning-rate)
180         scheduler.step(val_loss)
181

```

- In **Train/Valid Loss & Acc + LR hiện tại** để bạn nắm tiến trình (rất hữu ích khi debug).
- Cập nhật LR dựa trên `val_loss`.

```

183         if val_loss < best_valid_loss:
184             best_valid_loss = val_loss
185             patience_counter = 0
186             torch.save(model.state_dict(), model_path)
187             logging.info(f"Đã lưu mô hình tốt nhất cho {model_name} tại {model_path}")
188         else:
189             patience_counter += 1
190             if patience_counter >= patience:
191                 logging.info(f"Early stopping triggered for {model_name} (Best Loss: {best_valid_loss:.4f})")
192                 break
193

```

- Nếu `val_loss` **cải thiện** → reset `patience_counter` và **lưu checkpoint** (chỉ `state_dict`, gọn nhẹ).

- Nếu **không cải thiện** trong `patience` epoch liên tiếp → **dừng sớm** (tránh overfitting, tiết kiệm thời gian).
- **Lưu ý nhỏ:** tiêu chí “<” (nhỏ hơn) nghĩa là **bằng nhau** cũng tính là **không cải thiện** → hợp lý để thúc đẩy học tiếp/hạ LR.

```
194         return train_losses, valid_losses, train_accs, valid_accs
```

Dùng cho vẽ đồ thị học, so sánh mô hình, báo cáo.

Tổng thể:

- Xử lý lệch lớp:** `class_weights` trong `CrossEntropy`.
- Giảm overfitting:** `weight_decay` (L2), **early stopping**, và bạn có thể kết hợp **data augmentation** ở `preprocessing.py`.
- Ổn định tối ưu:** Adam + **ReduceLROnPlateau** (giảm LR khi `val_loss` phẳng).
- Tái lập & tái sử dụng:** lưu **best checkpoint** theo `val_loss`.
- Khả năng theo dõi:** log đầy đủ Loss/Acc & LR → để kiểm tra over/under-fitting.

Section 5.04 4) Hàm trực quan hóa

- Hàm `plot_learning_curves()` dùng để trực quan hóa quá trình huấn luyện mô hình học sâu thông qua hai biểu đồ:
- Loss curve (đường cong mất mát): cho thấy mô hình học được bao nhiêu qua từng epoch.
- Accuracy curve (độ chính xác): thể hiện khả năng mô hình dự đoán đúng theo thời gian.
- Đây là công cụ chính để phát hiện overfitting hoặc underfitting và giúp điều chỉnh lại hyperparameters (learning rate, dropout, regularization...).

- Khởi tạo và xác định nơi lưu hình

```
203         """Hàm vẽ learning curves (Loss và Accuracy)."""
204         os.makedirs(save_dir, exist_ok=True)
205         save_path = os.path.join(save_dir, f'{model_name}_learning_curves.png')
```

`os.makedirs(save_dir, exist_ok=True)`

→ Tạo thư mục `plots/` nếu chưa tồn tại, tránh lỗi khi lưu file.

`save_path`

→ Gộp tên mô hình (`model_name`) vào tên ảnh → ví dụ: `plots/ResNet18_learning_curves.png`.

→ Giúp bạn lưu nhiều biểu đồ riêng cho từng mô hình mà không bị đè.

- Tác dụng: bảo đảm pipeline chạy an toàn, không cần tạo thủ công thư mục trước.

- Tính số epoch

```
207         epochs = range(1, len(train_losses) + 1)
```

Số epoch tương ứng với độ dài danh sách train_losses.

Dùng làm trục X (Epochs) cho cả hai biểu đồ.

Tác dụng: đảm bảo đồ thị hiển thị chính xác theo số vòng huấn luyện thực tế (kể cả khi dừng sớm).

-Tạo Figure chính

```
209 | plt.figure(figsize=(12, 5))
```

- Tạo một **canvas chứa 2 biểu đồ con (subplot)**.
- Kích thước (12,5) giúp hiển thị vừa vặn cả Loss và Accuracy song song.

Biểu đồ 1 – Loss

```
211 | plt.subplot(1, 2, 1)
212 | plt.plot(epochs, train_losses, label='Train Loss')
213 | plt.plot(epochs, valid_losses, label='Valid Loss')
214 | plt.title(f'{model_name} Loss Curve')
215 | plt.xlabel('Epoch')
216 | plt.ylabel('Loss')
217 | plt.legend()
```

subplot(1,2,1) → vị trí đầu tiên (trái) trong figure.

-Vẽ 2 đường:

- train_losses: đường thể hiện **mức mất mát của tập huấn luyện** qua từng epoch.
- valid_losses: đường thể hiện **mức mất mát của tập kiểm định (validation)**.
- Gán tiêu đề, nhãn trục và chú thích (legend).
- Nếu cả Train và Valid Loss cùng giảm đều → mô hình học tốt.
- Nếu Train Loss ↓ còn Valid Loss ↑ → **Overfitting**.
- Nếu cả hai cao → **Underfitting** hoặc học chưa đủ lâu.

-Biểu đồ 2 – Accuracy

```
219 | plt.subplot(1, 2, 2)
220 | plt.plot(epochs, train_accs, label='Train Acc')
221 | plt.plot(epochs, valid_accs, label='Valid Acc')
222 | plt.title(f'{model_name} Accuracy Curve')
223 | plt.xlabel('Epoch')
224 | plt.ylabel('Accuracy (%)')
225 | plt.legend()
```

- Tương tự như trên nhưng thể hiện **độ chính xác (accuracy)** thay vì loss.
- Hiển thị tỉ lệ phần trăm % đúng qua từng epoch.
- Nếu Train Acc ↑ còn Valid Acc giảm → Overfitting.
- Nếu cả hai tăng đều và hội tụ gần nhau → mô hình học tốt, tổng quát hóa tốt.
- Nếu cả hai thấp và tăng chậm → mô hình còn underfit.

```

227 | plt.tight_layout()
228 | plt.savefig(save_path)
229 | plt.close()
230 | logging.info(f"Đã lưu learning curves cho {model_name} tại {save_path}")

```

- plt.tight_layout() → tự động căn chỉnh khoảng cách giữa các subplot, tránh chữ bị chồng lên.
- plt.savefig(save_path) → lưu biểu đồ ra file PNG (dễ nhúng vào báo cáo hoặc slide).
- plt.close() → giải phóng bộ nhớ để tránh lỗi khi vẽ nhiều mô hình.
- logging.info(...) → thông báo trên terminal để xác nhận file đã được lưu.
- Tác dụng: đảm bảo biểu đồ gọn gàng, dễ đọc, tiết kiệm RAM.

-Mục đích tổng quan:

```

242 | # Danh sách mô hình cần huấn luyện
243 | models_to_train: Dict[str, nn.Module] = {
244 |     'ResNet18': define_resnet18(num_classes, use_pretrained=True),
245 |     'MobileNetV2': define_mobilenetv2(num_classes, use_pretrained=True),
246 |     'EfficientNetB0': define_efficientnetb0(num_classes, use_pretrained=True),
247 | }

```

-Hàm run_model_comparison() là hàm tổng hợp và tự động hóa toàn bộ quá trình huấn luyện, đánh giá và so sánh giữa nhiều mô hình học sâu (Deep Learning) khác nhau — cụ thể là: ResNet18, MobileNetV2, EfficientNetB0

-Đây là bước đánh giá mô hình (Model Evaluation & Benchmarking) trong pipeline học sâu, giúp xác định mô hình nào hoạt động tốt nhất trên cùng một bộ dữ liệu bệnh lá cây (Plant Diseases Dataset).

-Cấu trúc tổng quát và ý nghĩa tham số

```

234 | def run_model_comparison(
235 |     train_loader: Any, valid_loader: Any, class_weights: torch.Tensor,
236 |     device: torch.device, num_classes: int, epochs: int = 50, patience: int = 5
237 | ) -> Dict[str, Dict[str, str]]:
238 |     """

```

Tham số	Ý nghĩa
train_loader, valid_loader	Bộ dữ liệu huấn luyện và kiểm định (đọc bằng DataLoader)
class_weights	Trọng số lớp để xử lý mất cân bằng dữ liệu
device	Thiết bị tính toán (cuda hoặc cpu)
num_classes	Số lớp bệnh (số nhãn đầu ra)
epochs	Số vòng huấn luyện tối đa
patience	Số epoch nhẫn nại trước khi dừng sớm (early stopping)
Kết quả trả về:	results: dictionary chứa thông kê độ chính xác và mất mát tốt nhất của từng mô hình

Tham số

Ý nghĩa

-Tạo danh sách mô hình cần huấn luyện

```
242     # Danh sách mô hình cần huấn luyện
243     models_to_train: Dict[str, nn.Module] = {
244         'ResNet18': define_resnet18(num_classes, use_pretrained=True),
245         'MobileNetV2': define_mobilenetv2(num_classes, use_pretrained=True),
246         'EfficientNetB0': define_efficientnetb0(num_classes, use_pretrained=True),
247     }
```

-Gọi ba hàm định nghĩa mô hình (từ phần đầu file) để khởi tạo sẵn 3 backbone pre-trained.

use_pretrained=True nghĩa là các mô hình dùng trọng số huấn luyện sẵn trên ImageNet → giúp học nhanh và chính xác hơn.

-num_classes được thay vào tầng cuối (Fully Connected layer) để phù hợp với dữ liệu lá cây (ví dụ 38 lớp bệnh).

-Tác dụng:

Giúp bạn huấn luyện đồng thời nhiều kiến trúc mạng CNN khác nhau để tìm ra mô hình có độ chính xác cao nhất và ít overfitting nhất.

-Vòng lặp qua từng mô hình

```
251     for model_name, model in models_to_train.items():
252         logging.info(f"\n--- Bắt đầu huấn luyện mô hình {model_name} ---")
253
```

-for model_name, model → duyệt qua từng mô hình trong danh sách.

-logging.info() → ghi log hiển thị mô hình đang được train.

- Tác dụng: giúp bạn theo dõi tiến trình huấn luyện từng mô hình riêng biệt (ResNet → MobileNet → EfficientNet).

-Huấn luyện mô hình và lấy lịch sử học

```
254     # Huấn luyện mô hình và lấy lịch sử Loss/Acc
255     train_losses, valid_losses, train_accs, valid_accs = train_model(
256         model, model_name, train_loader, valid_loader, class_weights, device,
257         epochs=epochs, patience=patience
258     )
```

-Gọi lại hàm train_model() đã được định nghĩa trước đó (hàm trung tâm của pipeline).

-Kết quả trả về 4 danh sách chứa loss và accuracy qua từng

epoch:train_losses,valid_losses,train_accs,valid_accs.-Tác dụng:

Thu thập dữ liệu học tập (learning history) để trực quan hóa và đánh giá quá trình huấn luyện.

-Vẽ biểu đồ học

```
260     # Vẽ và lưu Learning Curves
261     plot_learning_curves(train_losses, valid_losses, train_accs, valid_accs, model_name)
```


-Hàm `plot_learning_curves()` sẽ tạo và lưu hình biểu đồ (Loss + Accuracy) cho từng mô hình.

-Ghi nhận kết quả tốt nhất của từng mô hình

```
263         # Thu thập kết quả tốt nhất (cho mục đích báo cáo)
264         best_val_acc = max(valid_accs) if valid_accs else 0.0
265         best_val_loss = min(valid_losses) if valid_losses else float('inf')
266
267         results[model_name] = {
268             'best_val_acc': f"{best_val_acc:.2f}%",
269             'best_val_loss': f"{best_val_loss:.4f}",
270             'epochs_run': len(train_losses)
271         }
```

-`best_val_acc`: độ chính xác cao nhất của mô hình trên tập validation.

-`best_val_loss`: giá trị mất mát thấp nhất → càng nhỏ càng tốt.

-`epochs_run`: tổng số epoch mô hình thực sự chạy (trường hợp dừng sớm).

-Tác dụng:

-Lưu kết quả chi tiết cho mỗi mô hình, giúp bạn dễ dàng so sánh chất lượng và tốc độ hội tụ giữa các mô hình.

-In bảng kết quả tổng hợp cuối cùng

```
273         # In ra kết quả tóm tắt cuối cùng
274         logging.info("\n=====")
275         logging.info("--- KẾT QUẢ TÓM TẮT SO SÁNH CÁC MÔ HÌNH (DÙNG CHO BÁO CÁO) ---")
276         for model, res in results.items():
277             logging.info(f"| Model: {model.ljust(14)} | Best Val Acc: {res['best_val_acc'].ljust(6)} | Best
278             logging.info("=====")
279
```

- Cung cấp **bảng tổng kết định lượng** để bạn đưa vào báo cáo hoặc slide thuyết trình.

- Dễ dàng xác định mô hình nào tốt nhất.

-Trả về kết quả cuối cùng

```
280         return results
```

có thể dùng dữ liệu này để:

-Ghi vào file .csv / .json

-Vẽ biểu đồ so sánh giữa các mô hình

-Tự động chọn mô hình tốt nhất cho phần dự đoán sau.

-KIỂM TRA MÔI TRƯỜNG VÀ IMPORT HÀM TIỀN XỬ LÝ

```
285         try:
286             from preprocessing import get_device, load_datasets, get_dataloaders, compute_class_weights
287         except ImportError:
288             logging.error("Lỗi: Không tìm thấy preprocessing.py")
289             exit()
```

-Đảm nhiệm **vai trò khởi động hệ thống**, đảm bảo tất cả các hàm phụ trợ cần thiết cho pipeline huấn luyện đã sẵn sàng.

-Hàm `get_device()` giúp phát hiện GPU (nếu có CUDA) để tăng tốc xử lý; `load_datasets()` và `get_dataloaders()` dùng để đọc và chia batch dữ liệu; `compute_class_weights()` được sử dụng để xử lý vấn đề **mất cân bằng giữa các lớp bệnh lá** (một số lớp có nhiều ảnh hơn).

Cấu trúc `try-except` giúp chương trình **an toàn** — nếu thiếu file `preprocessing.py` hoặc hàm phụ, nó sẽ dừng lại có kiểm soát thay vì lỗi vỡ toàn bộ tiến trình.

→ **Ý nghĩa:** Đây là “bộ kiểm tra an toàn khởi động” cho pipeline, giúp môi trường huấn luyện sẵn sàng trước khi vào giai đoạn nặng (training GPU).

-Xác định dữ liệu và tải data

```
291     ds_dir = r"E:\New_Plant_Diseases_Project\data\New Plant Diseases Dataset(Augmented)\New Plant Diseases
292     train_dir = os.path.join(ds_dir, 'train')
293     valid_dir = os.path.join(ds_dir, 'valid')
294
295     device = get_device()
296     logging.info(f"Thiết bị: {device}")
```

-Chương trình xác định **đường dẫn đến tập dữ liệu và thiết bị huấn luyện**.

-Dữ liệu được chia sẵn thành hai thư mục `train` và `valid`, mỗi thư mục chứa nhiều thư mục con tương ứng với từng loại bệnh lá cây (ví dụ: “Apple__Black_rot”, “Corn__Healthy”...).

-Lệnh `get_device()` xác định việc dùng GPU hay CPU, đảm bảo mô hình chạy ở môi trường tối ưu.

```
299     train_ds, valid_ds = load_datasets(train_dir, valid_dir)
300     train_loader, valid_loader = get_dataloaders(train_ds, valid_ds, batch_size=32)
301     class_weights = compute_class_weights(train_ds)
302     num_classes = len(train_ds.classes)
```

-Đọc toàn bộ ảnh, gán nhãn dựa theo tên thư mục con, tạo batch dữ liệu 32 ảnh/lần, tính trọng số lớp và đếm tổng số lớp cần phân loại.

→ **Ý nghĩa:** Đây là “khâu nạp nhiên liệu” cho mô hình. Nếu dữ liệu không tải đúng, mọi bước sau sẽ vô nghĩa. Nhờ có `DataLoader`, chương trình có thể luân chuyển ảnh vào GPU hiệu quả, tránh nghẽn bộ nhớ.

-Nếu có lỗi (thiếu thư mục, sai tên, hỏng ảnh), chương trình tự động ghi log và dừng — giúp người dùng biết ngay dữ liệu gặp sự cố.

-Khai báo và hướng luyện mô hình CNN

```
309     # CHỈ CHẠY CNN THUẦN
310     logging.info("\n--- HUẤN LUYỆN CNN THUẦN ---")
311     cnn_model = define_cnn(num_classes=num_classes)
312     cnn_model.to(device)
```

-Hệ thống khởi tạo mô hình **CNN thuần (Convolutional Neural Network)** — không dùng trọng số có sẵn (pre-trained).

-Do đó, mạng phải học từ đầu toàn bộ đặc trưng của bệnh lá: hình dạng, màu sắc, đốm bệnh, gân lá,...

```

314     train_losses, valid_losses, train_accs, valid_accs = train_model(
315         model=cnn_model,
316         model_name="CNN",
317         train_loader=train_loader,
318         valid_loader=valid_loader,
319         class_weights=class_weights,
320         device=device,
321         epochs=50, # Tăng epochs vì CNN học từ đầu
322         lr=0.001,
323         patience=15
324     )

```

-Hàm train_model() (đã định nghĩa ở trên) sẽ:

-Duyệt từng batch ảnh để cập nhật trọng số.

-Tính toán Loss (CrossEntropy) và Accuracy trên tập train & validation.

-Tự động giảm Learning Rate khi mô hình hội tụ chậm (ReduceLROnPlateau).

-Kích hoạt Early Stopping nếu sau 15 epoch không cải thiện.

-Lưu lại checkpoint mô hình tốt nhất vào thư mục models/.

-Vì đây là CNN học từ đầu, số epoch được tăng lên 50 để mô hình có đủ thời gian hội tụ.

→ Ý nghĩa: Cụm này là trái tim của pipeline – mô hình thực sự “học” tại đây. Toàn bộ logic tối ưu trọng số, điều chỉnh tốc độ học, và kiểm soát overfitting đều diễn ra trong hàm này.

```

326     plot_learning_curves(train_losses, valid_losses, train_accs, valid_accs, "CNN")

```

-Hàm này vẽ hai biểu đồ song song:

-Đường cong mất mát (Loss)

-Đường cong độ chính xác (Accuracy)

→ Giúp người nghiên cứu dễ dàng quan sát mô hình có học ổn định hay bị overfitting.

-Hình được lưu trong thư mục plots/CNN_learning_curves.png.

```

328     best_acc = max(valid_accs)
329     best_loss = min(valid_losses)
330     logging.info("\n" + "="*60)
331     logging.info("HOÀN TẤT!")
332     logging.info(f"Best Valid Acc: {best_acc:.2f}%")
333     logging.info(f"Best Valid Loss: {best_loss:.4f}")
334     logging.info(f"Model: models/best_CNN.pth")
335     logging.info(f"Plot: plots/CNN_learning_curves.png")
336     logging.info("="*60)

```

-Kết quả này giúp người dùng nhanh chóng xem **mô hình đạt độ chính xác bao nhiêu** (ví dụ 96.4%) và **giá trị mất mát nhỏ nhất**, đồng thời biết mô hình đã được lưu ở đâu.

→ **Ý nghĩa:** Đây là “báo cáo tổng kết huấn luyện”. Phần này được chèn vào báo cáo chương “Kết quả và đánh giá mô hình” để chứng minh mô hình đạt hiệu suất mong muốn, có khả năng phân loại bệnh lá hiệu quả.

Article VI. 6.Đánh giá mô hình:

Section 6.01 Khởi tạo cấu hình

```
3  import torch
4  import torch.nn as nn
5  import numpy as np
6  import os
7  import matplotlib.pyplot as plt
8  import seaborn as sns
9  from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score
10 from torch.utils.data import DataLoader
11 import pandas as pd
12 import json
13 import logging
14 import time
15 from datetime import datetime
16
17 from preprocessing import get_device, load_single_dataset
18 from model_training import define_resnet18, define_mobilenetv2, define_cnn
19 from torchvision.models import efficientnet_b0
20
28 logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
29
30 # === THƯ MỤC ===
31 PLOTS_DIR = "plots"
32 REPORT_DIR = "reports"
33 os.makedirs(PLOTS_DIR, exist_ok=True)
34 os.makedirs(REPORT_DIR, exist_ok=True)
```

-Chuẩn bị môi trường đánh giá mô hình.

-Import PyTorch, sklearn metrics (accuracy, f1, report, confusion matrix), matplotlib/seaborn để vẽ, pandas/json/logging/time.

-Bật logging.basicConfig(...) → có log rõ ràng trong console.

-Thư mục:

PLOTS_DIR = "plots" để lưu 7 biểu đồ/mô hình.

REPORT_DIR = "reports" để lưu CSV/HTML/JSON/NPY.

Tạo sẵn thư mục (tránh lỗi không tồn tại).

Section 6.02 Nạp mô hình trọng số

```
36 # === LOAD MODEL ===
37 def load_model_and_weights(name: str, num_classes: int, device):
38     path = f"models/best_{name}.pth"
39     if name == "ResNet18":
40         model = define_resnet18(num_classes, use_pretrained=False)
41     elif name == "MobileNetV2":
42         model = define_mobilenetv2(num_classes, use_pretrained=False)
43     elif name == "EfficientNetB0":
44         model = efficientnet_b0(num_classes=num_classes)
45     elif name == "CNN":
46         model = define_cnn(num_classes)
47     else:
48         raise ValueError(f"Unsupported model: {name}")
49
50     if not os.path.exists(path):
51         logging.error(f"KHÔNG TÌM THẤY: {path}")
52         return None
53
54     model.load_state_dict(torch.load(path, map_location=device))
55     logging.info(f"ĐÃ GẮN: {path}")
56     return model.to(device).eval()
57
```

-Hàm: `load_model_and_weights(name, num_classes, device)`

-Luồng:

Chọn kiến trúc theo name:

ResNet18 → `define_resnet18(num_classes, use_pretrained=False)`

MobileNetV2 → `define_mobilenetv2(num_classes, use_pretrained=False)`

EfficientNetB0 → đang gọi `efficientnet_b0(num_classes=num_classes)` (lỗi — xem bên dưới)

CNN → `define_cnn(num_classes)`

-Tìm `models/best_{name}.pth`. Không có → log lỗi và trả None.

-`model.load_state_dict(torch.load(path, map_location=device))` → nạp trọng số vào kiến trúc vừa tạo.

-`model.to(device).eval()` → sẵn sàng suy luận.

-Vấn đề & cách sửa (quan trọng):

`efficientnet_b0(num_classes=...)` không đúng API của torchvision.

-Cách chuẩn:

```
from torchvision.models import efficientnet_b0
```

```
model = efficientnet_b0(weights=None)
```

```
in_f = model.classifier[1].in_features
```

```
model.classifier[1] = nn.Linear(in_f, num_classes)
```

Trong `__main__` bạn truyền cứng `num_classes=38` khi gọi `load_model_and_weights(...)`. Nên lấy từ dataset:

```
num_classes = len(ds.classes)
```

```
model = load_model_and_weights(name, num_classes, device)
```

-Suy luận, đo thời gian Hàm:

```
70 # === INFERENCE TIME ===
71 @torch.no_grad()
72 def measure_inference_time(model, loader, device, n=10):
73     model.eval()
74     times = []
75     for i, (x, _) in enumerate(loader):
76         if i >= n: break
77         x = x.to(device)
78         if device.type == "cuda": torch.cuda.synchronize()
79         s = time.time()
80         _ = model(x)
81         if device.type == "cuda": torch.cuda.synchronize()
82         times.append(time.time() - s)
83     return np.mean(times) * 1000 if times else 0
84
```

-`get_predictions(model, loader, device)` → trả `y_true`, `y_pred` cho toàn bộ loader.

-`measure_inference_time(model, loader, device, n=10)` → đo ms/batch (bình quân trên 10 batch đầu).

Chi tiết:

-Điều dùng `@torch.no_grad()` và `model.eval()` → đúng chuẩn suy luận, tiết kiệm VRAM.

-Với CUDA: có `torch.cuda.synchronize()` trước/sau để đo thời gian chính xác.

-Kết quả timing hiện trả ms/batch duy nhất.

-Gợi ý nâng cấp:

-Trả thêm ms/image cho công bằng giữa model:

```
ms_batch = np.mean(times)*1000
```

```
ms_img = ms_batch / batch_size
```

-Đưa `non_blocking=True` khi `.to(device)` nếu DataLoader có `pin_memory=True`.

Section 6.03 Biểu đồ báo cáo chi tiết

-Confusion Matrix — `plot_confusion_matrix(...)`


```

85 # === CONFUSION MATRIX ===
86 def plot_confusion_matrix(y_true, y_pred, classes, name):
87     cm = confusion_matrix(y_true, y_pred)
88     plt.figure(figsize=(16, 14))
89     sns.heatmap(cm, annot=False, fmt='d', cmap='Blues',
90                 xticklabels=classes, yticklabels=classes, cbar=True)
91     plt.title(f"Confusion Matrix {name}")
92     plt.xticks(rotation=90, fontsize=8); plt.yticks(fontsize=8)
93     plt.tight_layout()
94     path = f"{PLOTS_DIR}/{name}_cm.png"
95     plt.savefig(path, dpi=150, bbox_inches='tight'); plt.close()
96     logging.info(f"Saved CM: {path}")

```

-Vẽ heatmap lớn (16×14), không annotate từng ô (đỡ rối), có nhãn lớp theo classes.

-Lưu plots/{name}_cm.png.

-Lưu .NPY chung: np.save("reports/confusion_matrix.npy", cm) → bị ghi đè giữa các mô hình.

-Ở dưới __main__ bạn lại lưu thêm: reports/confusion_matrix_{name}.npy (theo từng model) → cái chung phía trên là thừa & gây nhầm.

- Khuyến nghị: bỏ file chung, chỉ lưu per-model:

trong plot_confusion_matrix: bỏ dòng np.save("confusion_matrix.npy", cm)

và giữ np.save(f"{REPORT_DIR}/confusion_matrix_{name}.npy", cm) ở __main__

GradCAM++ — save_grad-cam(...)

```
117 # === GRADCAM ===
118 def save_gradcam(model, loader, device, name):
119     if not GRADCAM_AVAILABLE:
120         logging.warning("pytorch-grad-cam không có → bỏ qua GradCAM")
121         return
122
123     model.eval()
124     img, _ = next(iter(loader))
125     img = img[:1].to(device)
126
127     target_layer = get_target_layer(model, name)
128     if target_layer is None:
129         logging.warning(f"Không tìm thấy layer phù hợp cho GradCAM: {name}")
130         return
131
132     try:
133         cam = GradCAMPlusPlus(model=model, target_layers=[target_layer])
134         cam_map = cam(input_tensor=img)[0]
135
136         mean = np.array([0.485, 0.456, 0.406])
137         std = np.array([0.229, 0.224, 0.225])
138         img_np = img[0].cpu().numpy().transpose(1, 2, 0)
139         img_np = std * img_np + mean
140         img_np = np.clip(img_np, 0, 1)
141
142         overlay = show_cam_on_image(img_np, cam_map, use_rgb=True)
143         path = f"{PLOTS_DIR}/{name}_gradcam.png"
144         plt.imsave(path, overlay)
145         logging.info(f"GradCAM++: {path}")
146     except Exception as e:
147         logging.error(f"Lỗi GradCAM cho {name}: {e}")
148
```

-Lấy 1 ảnh đầu tiên từ loader → tính heatmap bằng GradCAM++ lên layer cuối của backbone.

-Khôi phục ảnh về [0,1] bằng mean/std ImageNet trước khi show_cam_on_image.

-Lưu plots/{name}_gradcam.png.

- F1 per class — plot_f1_per_class(...)

```
149 # === F1 PER CLASS ===
150 def plot_f1_per_class(report_dict, class_names, name):
151     f1_scores = [report_dict[c]['f1-score'] for c in class_names if c in report_dict]
152     plt.figure(figsize=(14, 6))
153     bars = plt.bar(range(len(f1_scores)), f1_scores, color='skyblue', edgecolor='navy', alpha=0.7)
154     plt.title(f"F1-Score Per Class {name}", fontsize=14)
155     plt.ylim(0.9, 1.0); plt.xticks([])
156     plt.grid(axis='y', linestyle='--', alpha=0.7)
157     min_idx = np.argmin(f1_scores)
158     bars[min_idx].set_color('red')
159     plt.text(min_idx, f1_scores[min_idx] + 0.001, class_names[min_idx],
160             ha='center', va='bottom', fontsize=8, color='red', fontweight='bold')
161     path = f"{PLOTS_DIR}/{name}_f1_per_class.png"
162     plt.tight_layout(); plt.savefig(path, dpi=150, bbox_inches='tight'); plt.close()
163     logging.info(f"F1 per class: {path}")

```

-Vẽ cột F1 theo lớp; tô đỏ lớp thấp nhất → rất hữu ích để phát hiện lớp khó.

- Nhược điểm nhỏ: bạn ép ylim(0.9–1.0) → nếu có lớp F1 < 0.9 sẽ bị cắt.
- Nên auto-scale theo min/max thực tế (± 0.05), hoặc đặt ylim(0,1).

-Precision vs Recall — plot_precision_recall_scatter(...)

```

165 # === PRECISION vs RECALL ===
166 def plot_precision_recall_scatter(report_dict, class_names, name):
167     precisions = [report_dict[c]['precision'] for c in class_names if c in report_dict]
168     recalls = [report_dict[c]['recall'] for c in class_names if c in report_dict]
169     plt.figure(figsize=(8, 8))
170     plt.scatter(recalls, precisions, c='teal', alpha=0.6, edgecolors='black', s=60)
171     plt.title(f"Precision vs Recall {name}")
172     plt.xlim(0.8, 1.01); plt.ylim(0.8, 1.01)
173     plt.plot([0.8, 1], [0.8, 1], 'r--', label="Perfect")
174     plt.grid(True, linestyle='--', alpha=0.5); plt.legend()
175     path = f"{PLOTS_DIR}/{name}_precision_recall.png"
176     plt.tight_layout(); plt.savefig(path, dpi=150, bbox_inches='tight'); plt.close()
177     logging.info(f"Precision-Recall: {path}")
178 
```

-Scatter (recall, precision) từng lớp, đường chéo “Perfect”.

- Bạn fix trục 0.8–1.01 → nếu lớp yếu <0.8 sẽ bị cắt.
- Đổi thành 0–1.01 hoặc auto-scale.

-Top-10 Easy/Hard — plot_top10_classes(...)

```

179 # === TOP 10 EASY/HARD ===
180 def plot_top10_classes(report_dict, class_names, name):
181     f1_scores = {c: report_dict[c]['f1-score'] for c in class_names if c in report_dict}
182     sorted_f1 = sorted(f1_scores.items(), key=lambda x: x[1], reverse=True)
183     easy = sorted_f1[:10]; hard = sorted_f1[-10:][::-1]
184     plt.figure(figsize=(12, 5))
185     plt.subplot(1, 2, 1); plt.barh([x[0] for x in easy], [x[1] for x in easy], color='lightgreen')
186     plt.title("Top 10 Easy"); plt.xlim(0.98, 1); plt.xlabel("F1")
187     plt.subplot(1, 2, 2); plt.barh([x[0] for x in hard], [x[1] for x in hard], color='salmon')
188     plt.title("Top 10 Hard"); plt.xlim(0.9, 1); plt.xlabel("F1")
189     plt.tight_layout()
190     path = f"{PLOTS_DIR}/{name}_top10_classes.png"
191     plt.savefig(path, dpi=150, bbox_inches='tight'); plt.close()
192     logging.info(f"Top 10: {path}")

```

-Tách top-10 F1 cao nhất (Easy) & thấp nhất (Hard).

-Cực hợp để thảo luận nguyên nhân nhầm lẫn trong báo cáo.

-Classification Report JSON:

classification_report(..., output_dict=True) → lưu JSON chi tiết cho từng lớp (precision/recall/f1/support) → dùng được cho web/dashboard.

Section 6.04 Bảng so sánh cuối lưu trữ

```
194 # === SO SÁNH MÔ HÌNH ===
195 def save_comparison_table(results):
196     df = pd.DataFrame(results).round(4)
197     print("\n" + "="*100)
198     print("FINAL MODEL COMPARISON")
199     print("="*100)
200     print(df.to_string(index=False))
201     df.to_csv(f"{REPORT_DIR}/model_comparison.csv", index=False)
202     df.to_html(f"{REPORT_DIR}/model_comparison.html", index=False)
203     best = df.loc[df['Accuracy'].idxmax()]
204     logging.info(f"BEST MODEL: {best['Model']} (Acc: {best['Accuracy']:.4f})")
205     return best['Model']
```

-Hàm: save_comparison_table(results)

-Nhận list dict: Model, Accuracy, F1-Score, Time (ms/batch), Size (MB) (hiện chưa có ms/image).

-In bảng ra console, lưu reports/model_comparison.csv & .html.

-Chọn BEST theo Accuracy.(Bạn có thể tie-break bằng F1 nếu acc bằng nhau.)

-Ở __main__:Lưu tổng hợp vào reports/evaluation_report.json gồm:best_model, results (array các model với metrics)

Pipeline chính


```

207 # === MAIN ===
208 if __name__ == "__main__":
209     valid_dir = r"E:\New_Plant_Diseases_Project\data\New Plant Diseases Dataset(Augmented)\New P
210     device = get_device()
211     ds = load_single_dataset(valid_dir)
212     loader = DataLoader(ds, batch_size=32, shuffle=False, num_workers=4, pin_memory=True)
213     class_names = ds.classes
214     logging.info(f"Dataset: {len(ds)} samples, {len(class_names)} classes")
215     available = [m for m in ["ResNet18", "MobileNetV2", "EfficientNetB0", "CNN"]
216                 if os.path.exists(f"models/best_{m}.pth")]
217     if not available:
218         logging.error("Không tìm thấy file .pth nào!")
219         exit()
220     logging.info(f"Tìm thấy {len(available)} mô hình: {available}")
221     results = []
222     for name in available:
223         logging.info(f"\n{' '*40} EVALUATING {name} {' '*40}")
224         model = load_model_and_weights(name, 38, device)
225         if model is None:
226             continue
227         t = measure_inference_time(model, loader, device)
228         y_true, y_pred = get_predictions(model, loader, device)
229         acc = accuracy_score(y_true, y_pred)
230         f1 = f1_score(y_true, y_pred, average='weighted')
231         # === BÁO CÁO CHI TIẾT ===
232         report = classification_report(y_true, y_pred, target_names=class_names, output_dict=True)
233         print(classification_report(y_true, y_pred, target_names=class_names, digits=4))
234         # === LƯU REPORT CHI TIẾT ===
235         with open(f"{REPORT_DIR}/classification_report_{name}.json", "w", encoding="utf-8") as f:
236             json.dump(report, f, indent=2, ensure_ascii=False)
237         # === LƯU CONFUSION MATRIX .NPY ===
238         cm = plot_confusion_matrix(y_true, y_pred, class_names, name)
239         np.save(f"{REPORT_DIR}/confusion_matrix_{name}.npz", cm)
240         # === BIỂU ĐỒ ===
241         save_gradcam(model, loader, device, name)
242         plot_f1_per_class(report, class_names, name)
243         plot_precision_recall_scatter(report, class_names, name)
244         plot_top10_classes(report, class_names, name)
245         size_mb = round(os.path.getsize(f"models/best_{name}.pth") / (1024*1024), 2)
246         results.append({
247             "Model": name,
248             "Accuracy": acc,
249             "F1-Score": f1,
250             "Time (ms/batch)": round(t, 2),
251             "Size (MB)": size_mb
252         })
253     best_model = save_comparison_table(results)

```

-Luồng:

-Đặt valid_dir (đường dẫn tập validation).

-device = get_device(); load dataset validation (transform không augment) bằng load_single_dataset.

-DataLoader(ds, batch=32, shuffle=False, num_workers=4, pin_memory=True).

-Liệt kê model nào có checkpoint trong models/best_*.pth.

-Với mỗi model:

+ Nạp kiến trúc + weights (load_model_and_weights).

+ Đo thời gian suy luận (ms/batch).

+ Suy luận toàn bộ → `y_true`, `y_pred` → tính `acc`, `f1_weighted` → xuất `classification_report` (console JSON).

+ Vẽ & lưu: CM, GradCAM, F1 per class, Precision–Recall, Top-10.

+ Tính kích thước checkpoint (MB).

+ Đưa vào results.

-In & lưu bảng so sánh; lưu `evaluation_report.json`.

-Điểm mạnh tổng thể:

+ Một lệnh chạy → có đủ số liệu & biểu đồ & báo cáo cho mỗi mô hình.

+ Bao quát độ chính xác, diễn giải (GradCAM), chi phí (thời gian, dung lượng).

+ Dùng riêng validation không augment → đánh giá khách quan.

-Những chỗ nên chỉnh ngay:

+ EfficientNetB0: thay cách tạo head như đã nêu.

+ Confusion Matrix .npy: bỏ file chung `confusion_matrix.npy` trong `plot_confusion_matrix`, chỉ giữ bản `confusion_matrix_{name}.npy` theo từng model.

+ Hard-code `num_classes=38`: thay bằng `len(ds.classes)` khi gọi `load_model_and_weights`.

+ GradCAM cho CNN: tìm conv cuối bằng `model.modules()` thay vì `reversed(model)`.

+ Giới hạn trục biểu đồ: bỏ fix `ylim(0.9,1.0)` (F1) & `xlim/ylim(0.8,1.01)` (P/R) → auto-scale hoặc 0–1.

+ Timing: thêm `ms/image` để so sánh công bằng giữa batch size khác nhau.

Patch code gợi ý (chèn nhanh)

EfficientNetB0:

```
from torchvision.models import efficientnet_b0
```

```
# ...
```

```
elif name == "EfficientNetB0":
```

```
    model = efficientnet_b0(weights=None)
```

```
    in_f = model.classifier[1].in_features
```

```
    model.classifier[1] = nn.Linear(in_f, num_classes)
```

Gọi `num_classes` đúng:

```
num_classes = len(class_names)
```

```
model = load_model_and_weights(name, num_classes, device)
```

GradCAM – CNN target layer:

```
def get_target_layer(model, name: str):
```

```
    lname = name.lower()
```

```
    if "resnet" in lname:
```

```
        return model.layer4[-1]
```

```
    if "mobilenet" in lname or "efficientnet" in lname:
```

```

    return model.features[-1]
# generic fallback: last Conv2d in modules
for m in reversed(list(model.modules())):
    if isinstance(m, nn.Conv2d):
        return m
return None

```

Confusion Matrix lưu per-model (bỏ file chung):

Xóa dòng `np.save(f'{REPORT_DIR}/confusion_matrix.npy', cm)` trong `plot_confusion_matrix`.

Ở `__main__` bạn đã có:

```
np.save(f'{REPORT_DIR}/confusion_matrix_{name}.npy', cm)
```

→ giữ nguyên.

F1 per class auto-scale:

```
plt.ylim(max(0.0, min(f1_scores)-0.05), min(1.0, max(f1_scores)+0.05))
```

Precision–Recall axis:

```
plt.xlim(0.0, 1.01); plt.ylim(0.0, 1.01)
```

Timing ms/image:

```
t_batch = measure_inference_time(...)
```

```
ms_per_image = t_batch / 32 # nếu batch=32
```

Article VII. 7. Dự đoán bằng hình ảnh

Import thư viện

```

1  # api/app.py - ĐÃ SỬA LỖI ĐƯỜNG DẪN TUYỆT ĐỐI
2
3  import sys
4  import os
5  from flask import Flask, request, jsonify
6  from flask_cors import CORS
7  from PIL import Image
8  import torch
9  import torchvision.transforms as transforms
10 import numpy as np
11 import io
12 import base64

```


Dòng Thư viện Công dụng

Dòng	Thư viện	Công dụng
1–2	sys, os	Làm việc với đường dẫn file, thêm thư mục cha vào sys.path để import module dễ dàng.
3-4	Flask, CORS	Flask dùng để tạo API web; CORS cho phép frontend (React Native / web) truy cập API từ domain khác.
5	PIL.Image	Dùng để xử lý ảnh (mở, chuyển định dạng, resize, v.v.)
6	torch	Thư viện PyTorch — để load mô hình deep learning.
7	torchvision.transforms	Tiền xử lý ảnh (resize, normalize, convert tensor).
8	numpy	Xử lý ma trận, dữ liệu số.
9	i o	Chuyển dữ liệu ảnh dạng bytes thành ảnh đọc được.
10	base64	Dùng để mã hóa/giải mã ảnh (thường gửi từ client dưới dạng base64).

Section 7.01 Thiết lập đường dẫn tuyệt đối

```
14 # --- THIẾT LẬP ĐƯỜNG DẪN DỰ ÁN VÀ THƯ MỤC DATA ---
15 # 1. Xác định thư mục gốc của dự án (thư mục cha của 'api/')
16 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
17 sys.path.append(BASE_DIR)
18
```

- `__file__`: đường dẫn của file hiện tại (`api/app.py`)
- `os.path.abspath(__file__)`: chuyển sang đường dẫn tuyệt đối (VD: `C:/CNTT_TLUS/Project/api/app.py`)
- `os.path.dirname(...)` hai lần → lùi lên **2 cấp thư mục** để lấy thư mục gốc của dự án (VD: `C:/CNTT_TLUS/Project`)
- `sys.path.append(BASE_DIR)` giúp Python có thể **import module từ thư mục gốc**, ví dụ import file trong `src/` hay `models/`.

Thiết lập thư mục dữ liệu dự phòng

```
19 # 2. Định nghĩa các đường dẫn tương đối
20 VALID_DATA_PATH_FALLBACK = os.path.join(
21     BASE_DIR,
22     "data",
23     "New Plant Diseases Dataset(Augmented)",
24     "New Plant Diseases Dataset(Augmented)",
25     "valid"
26 )
```

- Xây dựng đường dẫn tuyệt đối đến thư mục `valid` chứa dữ liệu validation.
- `os.path.join` nối các thành phần đường dẫn lại an toàn, phù hợp mọi hệ điều hành.
- `VALID_DATA_PATH_FALLBACK` nghĩa là: “nếu không chỉ định rõ thư mục dữ liệu thì mặc định lấy thư mục này”.

Đường dẫn đến thư mục mô hình

```
27 MODELS_DIR = os.path.join(BASE_DIR, "models")
```

- Tạo biến trỏ đến thư mục chứa file mô hình đã huấn luyện (VD: resnet50_leaf.pth, mobilenet_leaf.pth, ...)
- Mục tiêu: Flask API sẽ load mô hình từ đây để dự đoán ảnh bệnh lá gửi từ client.

Kiểm tra thư mục valid_dir

```

29 # 3. Lấy valid_dir từ biến môi trường (set bởi run.py) hoặc dùng fallback
30 valid_dir = os.getenv("VALID_DIR")
31 if not valid_dir or not os.path.exists(valid_dir):
32     valid_dir = VALID_DATA_PATH_FALLBACK
33     # Kiểm tra lần cuối
34     if not os.path.exists(valid_dir):
35         raise FileNotFoundError(f"Không tìm thấy thư mục 'valid'! Đã thử {VALID_DATA_PATH_FALLBACK} và VALID_DIR env.")

```

- os.getenv("VALID_DIR") lấy biến môi trường VALID_DIR (nếu bạn đã set trước bằng run.py hoặc trên hệ thống).
- Nếu không có hoặc thư mục đó không tồn tại, nó sẽ dùng giá trị dự phòng VALID_DATA_PATH_FALLBACK (được định nghĩa ở phần trước).
- Sau đó kiểm tra lại một lần cuối — nếu vẫn không tìm thấy thư mục valid, nó báo lỗi FileNotFoundError.

Import các thành phần trong dự án

```

38 # Import từ dự án (đã thêm BASE_DIR vào sys.path)
39 from src.model_training import define_resnet18, define_mobilenetv2
40 # HÀM load_model_and_weights ĐÃ ĐƯỢC CẬP NHẬT Ở evaluation.py
41 from src.evaluation import load_model_and_weights
42 from src.preprocessing import load_single_dataset

```

- define_resnet18 và define_mobilenetv2: hàm định nghĩa cấu trúc mô hình CNN (kiến trúc ResNet18, MobileNetV2).
- load_model_and_weights: hàm trong evaluation.py để load mô hình và trọng số .pth đã huấn luyện xong.
- load_single_dataset: hàm trong preprocessing.py để tạo DataLoader hoặc dataset từ thư mục valid.

Import Grad-CAM (visualization)

```
44 from pytorch_grad_cam import GradCAMPlusPlus
45 from pytorch_grad_cam.utils.image import show_cam_on_image
```

-Hai dòng này dùng để tạo và hiển thị bản đồ Grad-CAM, giúp trực quan hóa vùng ảnh mà mô hình “chú ý” khi dự đoán.

Khởi tạo Flask app

```
47 app = Flask(__name__)
48 CORS(app)
```

- Flask(__name__): khởi tạo ứng dụng Flask, biến app là server chính.
- CORS(app): bật Cross-Origin Resource Sharing để cho phép client (chẳng hạn app React Native) gọi API Flask từ domain khác mà không bị lỗi CORS.

Kiểm tra thiết bị (GPU/CPU)

```
50 # === LOAD MODELS ===
51 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
52 print(f"Using device: {device}")
--
```

- Nếu máy có GPU (CUDA), mô hình sẽ chạy trên GPU để nhanh hơn.
- Nếu không, nó tự động chuyển sang CPU.
- Dòng print chỉ để in ra thiết bị hiện đang được dùng.

Load dataset và lớp nhần

```

54 # Load class names
55 dataset = load_single_dataset(valid_dir)
56 class_names = dataset.classes
57 print(f"Loaded {len(class_names)} classes")

```

- Gọi hàm `load_single_dataset(valid_dir)` để load dữ liệu ảnh từ thư mục `valid`.
- Lấy danh sách tên lớp (labels) từ `dataset.classes`, ví dụ: ['Apple___Scab', 'Corn___Gray_Leaf_Spot', ...].
- In ra số lượng lớp (ví dụ 38 lớp bệnh lá).

(a) Load mô hình đã huấn luyện

```

59 # Load models (TRUYỀN models_dir ĐỂ ĐẢM BẢO TÌM ĐƯỢC FILE .pth)
60 resnet = load_model_and_weights("ResNet18", 38, device, models_dir=MODELS_DIR)
61 mobilenet = load_model_and_weights("MobileNetV2", 38, device, models_dir=MODELS_DIR)

```

- Hai hàm này sẽ load mô hình ResNet18 và MobileNetV2 từ thư mục `models_dir` (đường dẫn được định nghĩa ở phần trước).
- Tham số "38" chính là số lớp bệnh (output classes).
 - `device` giúp chuyển mô hình sang GPU hoặc CPU tùy máy.
 - Nếu file trọng số `.pth` chưa tồn tại (chưa huấn luyện xong hoặc chưa lưu), dòng sau sẽ báo lỗi:

```

62 if resnet is None or mobilenet is None:
63     raise RuntimeError("Không thể tải mô hình. Vui lòng chạy evaluation.py trước!")

```

Phần transform – Tiền xử lý ảnh

```

66 # Transform
67 transform = transforms.Compose([
68     transforms.Resize((224, 224)),
69     transforms.ToTensor(),
70     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
71 ])

```

- Đây là pipeline tiền xử lý ảnh – rất quan trọng để ảnh đầu vào có định dạng giống lúc huấn luyện mô hình.

- `Resize((224, 224))`: đổi kích thước ảnh về 224×224 pixel (chuẩn cho ResNet, MobileNet).
- `ToTensor()`: chuyển ảnh PIL (RGB 0–255) thành tensor PyTorch có giá trị 0–1.
- `Normalize(mean, std)`: chuẩn hóa ảnh theo thống kê của tập ImageNet — giúp mô hình hoạt động ổn định hơn.
 - `mean = [0.485, 0.456, 0.406]`
 - `std = [0.229, 0.224, 0.225]`
 (mỗi giá trị tương ứng với kênh R, G, B)

Hàm `predict_and_gradcam()` – Dự đoán và sinh bản đồ Grad-CAM

```

73  # === PREDICT FUNCTION (GIỮ NGUYÊN LOGIC) ===
74  def predict_and_gradcam(model, img_pil, target_layer):
75      img_tensor = transform(img_pil).unsqueeze(0).to(device)
76

```

- Hàm này nhận vào:

- `model`: mô hình deep learning (ResNet hoặc MobileNet)
- `img_pil`: ảnh PIL (ảnh gốc gửi từ client)
- `target_layer`: lớp trong mạng dùng để vẽ Grad-CAM (thường là layer convolution cuối cùng)

- `transform(img_pil)`: áp dụng tiền xử lý ảnh.

- `.unsqueeze(0)`: thêm 1 chiều batch → (1, 3, 224, 224).

- `.to(device)`: chuyển tensor sang GPU hoặc CPU.

Dự đoán lớp bệnh

```

77      with torch.no_grad():
78          output = model(img_tensor)
79          prob = torch.softmax(output, dim=1)
80          confidence, pred_idx = torch.max(prob, dim=1)
81          disease = class_names[pred_idx.item()]
82          confidence = confidence.item() * 100
83

```

- `torch.no_grad()`: tắt chế độ tính gradient → tiết kiệm bộ nhớ khi dự đoán.

- `output = model(img_tensor)`: chạy ảnh qua mô hình.

- `torch.softmax`: chuyển đầu ra sang xác suất cho từng lớp bệnh.

- `torch.max`: lấy xác suất cao nhất và chỉ số tương ứng (`pred_idx`).

- disease: tên bệnh tương ứng với chỉ số đó.
- confidence: độ tin cậy của dự đoán, đổi sang phần trăm (%).

Tạo bản đồ Grad-CAM

```
84 | # GradCAM
85 | cam = GradCAMPlusPlus(model=model, target_layers=[target_layer])
86 | grayscale_cam = cam(input_tensor=img_tensor)[0]
```

- GradCAMPlusPlus: phương pháp sinh bản đồ nhiệt (heatmap) thể hiện vùng mô hình quan tâm.
- target_layers: chỉ lớp cần phân tích (thường là layer cuối của CNN).
- cam(input_tensor=img_tensor): chạy Grad-CAM để tạo bản đồ grayscale.
- [0]: lấy ảnh đầu tiên trong batch.

Chuyển đổi và hiển thị overlay Grad-CAM

```
88 | mean = np.array([0.485, 0.456, 0.406])
89 | std = np.array([0.229, 0.224, 0.225])
90 | img_np = img_tensor[0].cpu().numpy().transpose(1, 2, 0)
91 | img_np = std * img_np + mean
92 | img_np = np.clip(img_np, 0, 1)
```

- Chuyển tensor về numpy để hiển thị.
- Đảo lại thứ tự trục (C, H, W) $\rightarrow$ (H, W, C) để khớp với ảnh RGB.
- “Giải chuẩn hóa” ảnh: nhân lại std, cộng mean để phục hồi màu gốc.
- Dùng clip để giữ giá trị trong [0, 1].

Tạo ảnh chồng giữa ảnh gốc và Grad-CAM

```
94 | overlay = show_cam_on_image(img_np, grayscale_cam, use_rgb=True)
95 | overlay_pil = Image.fromarray((overlay * 255).astype(np.uint8))
```

- show_cam_on_image: chồng bản đồ nhiệt lên ảnh gốc để thấy vùng mô hình chú ý.

- Ảnh kết quả được chuyển về định dạng uint8 (0–255) rồi sang kiểu PIL.Image để lưu hoặc hiển thị.

Mã hóa ảnh để gửi về client

```
97     buffered = io.BytesIO()
98     overlay_pil.save(buffered, format="PNG")
99     img_str = base64.b64encode(buffered.getvalue()).decode()
100     return disease, round(confidence, 2), f"data:image/png;base64,{img_str}"
```

- io.BytesIO() tạo vùng nhớ ảo để lưu ảnh tạm thời.
- overlay_pil.save() lưu ảnh Grad-CAM vào vùng nhớ đó dưới dạng PNG.
- base64.b64encode() mã hóa ảnh thành chuỗi Base64 — giúp gửi qua JSON.
- Cuối cùng hàm trả về:
 1. disease: tên bệnh được dự đoán.
 2. confidence: độ tin cậy (%).
 3. img_str: chuỗi base64 của ảnh Grad-CAM (client có thể hiển thị ngay).

Định nghĩa route

```
102     # === API ===
103     @app.route("/predict", methods=["POST"])
104     def predict():
```

🕒 @app.route("/predict", methods=["POST"]) tạo **API endpoint** tại địa chỉ /predict, chỉ chấp nhận phương thức **POST**.

🕒 Khi client (web hoặc mobile app) gửi ảnh lên, Flask sẽ gọi hàm predict() này.

Kiểm tra file ảnh gửi lên

```
105     if 'image' not in request.files:
106         return jsonify({"error": "No image"}), 400
```

- request.files chứa các file mà client gửi lên (qua form-data).

- Nếu không có file tên 'image', trả về lỗi JSON "No image" cùng mã lỗi **400 – Bad Request**.

Đọc ảnh và chuyển sang RGB

```
108 | file = request.files['image']
109 | img_pil = Image.open(file.stream).convert("RGB")
```

- Lấy file ảnh từ form (request.files['image']).

- Image.open(file.stream) đọc luồng dữ liệu file và tạo đối tượng ảnh PIL.

- .convert("RGB") đảm bảo ảnh có 3 kênh màu (tránh lỗi nếu ảnh là PNG với 4 kênh RGBA).

=> img_pil chính là ảnh đã được chuẩn hóa để đưa vào model.

Lấy tên mô hình từ form (mặc định ResNet18)

```
111 | model_name = request.form.get("model", "ResNet18")
```

- request.form.get() lấy giá trị model từ form-data mà client gửi kèm.

- Nếu client không gửi, mặc định dùng "ResNet18".

- Như vậy người dùng có thể chọn mô hình "MobileNetV2" hoặc "ResNet18" để dự đoán.

(b) Chọn mô hình và layer phù hợp

```
112 | if model_name == "MobileNetV2":
113 |     model = mobilenet
114 |     layer = model.features[-1]
115 |     acc = 99.86
116 | else:
117 |     model = resnet
118 |     layer = model.layer4[-1]
119 |     acc = 99.91
```

- Nếu chọn "MobileNetV2" → dùng mô hình mobilenet (đã load trước đó), lấy **layer cuối cùng trong features** để tạo Grad-CAM.

- Ngược lại → dùng resnet, layer cuối cùng của layer4.

- acc là độ chính xác mô hình đã đạt được trong quá trình huấn luyện (hiển thị cho người dùng tham khảo).

Gọi hàm dự đoán và sinh Grad-CAM

```
121 | disease, confidence, gradcam_b64 = predict_and_gradcam(model, img_pil, layer)
```

- Gọi hàm `predict_and_gradcam` (đã giải thích ở phần trước).
- Hàm trả về 3 giá trị:
 1. `disease`: tên bệnh lá dự đoán.
 2. `confidence`: độ tin cậy (%).
 3. `gradcam_b64`: chuỗi base64 của ảnh Grad-CAM (để client hiển thị).

Trả kết quả JSON cho client

```
123 |         return jsonify({
124 |             "disease": disease,
125 |             "confidence": confidence,
126 |             "gradcam": gradcam_b64,
127 |             "model": model_name,
128 |             "accuracy": acc
129 |         })
```

- `jsonify` biến Python dict thành JSON trả về client.
- Gồm 5 trường:
 - + `"disease"`: tên bệnh (ví dụ `"Apple___Scab"`).
 - + `"confidence"`: độ tin cậy (ví dụ `97.43`).
 - + `"gradcam"`: ảnh Grad-CAM dạng base64 để hiển thị trực tiếp.
 - + `"model"`: mô hình đã dùng (ResNet18 hoặc MobileNetV2).
 - + `"accuracy"`: độ chính xác tổng thể của mô hình khi test.

Chạy Flask server

```
131 if __name__ == "__main__":  
132     app.run(host="0.0.0.0", port=5000, debug=True)
```

- Khi chạy file trực tiếp (python app.py), Flask khởi động web server trên:

+ **Host:** 0.0.0.0 → cho phép truy cập từ tất cả IP trong mạng.

+ **Port:** 5000 (mặc định của Flask).

+ **debug=True** → tự động reload khi code thay đổi và hiển thị lỗi chi tiết.

8.Hướng dẫn cách chạy trên local

